

**This document is for the users that use high
performance computing resources in UNIST
Supercomputing Center.
It starts on page 44 for the foreign researchers.**

<목 차>

1. HPC Resources	3	3. Support Apps user-guide	15
A. Configuration Overview	3	A. Environment Settings	15
B. Systems configuration diagram	4	i. Overview	15
C. How to access HPC systems	5	ii. Module commands	15
D. Disk quota	5	B. Conda Environment settings(using miniconda)	16
E. Hardware Spec (HPC Dumbo Cluster)	6	i. Overview	16
i. System Overview	6	ii. Module commands	16
1) GPU Nodes	6	iii. 사용자 폴더에서의 conda 환경 사용법(설정)	18
A) A100 GPU Nodes	6	C. Jupyter Notebook Settings	19
F. Software	7	i. SSH tunneling 설정 개요	19
i. Compilers	7	ii. SSH tunneling 설정 (putty 기준 설정)	20
ii. Libraries	7	iii. SSH tunneling 설정 (Xshell의 경우)	21
2. SLURM Schedule(for Job Control)	8	iv. 내PC 쪽(Windows/Mac) 브라우저 프록시 설정 (Firefox 기준 설명)	23
A. Overview	8	v. 터널링 설정 후 UNIST 슈퍼컴퓨터(리눅스) 접속 후 설정	24
B. SLURM Commands	8	vi. 터널링 설정 후 내PC에서 브라우저 접속 (Firefox 기준 설명)	27
C. SLURM partition information	9	vii. Jupyter Notebook 환경 확인 및 종료	29
D. sinfo (show computing node & partition status)	10	D. Nvidia-Docker	31
E. sbatch (submit a job)	11	i. Nvidia-Docker 스크립트 예제	31
F. squeue (작업목록 정보 조회)	13	ii. Nvidia-Docker Interactive mode	32
G. scontrol (제출된 작업 상세 조회)	14	4. 작업 스크립트 예제	34
F. scancel (cancel a job)	14	i. 작업 스크립트 예제(요약)	34
		ii. 작업 스크립트 예제(상세)	36

UNIST HPC-GPU System User Guide(Beta version)

1. HPC Resources

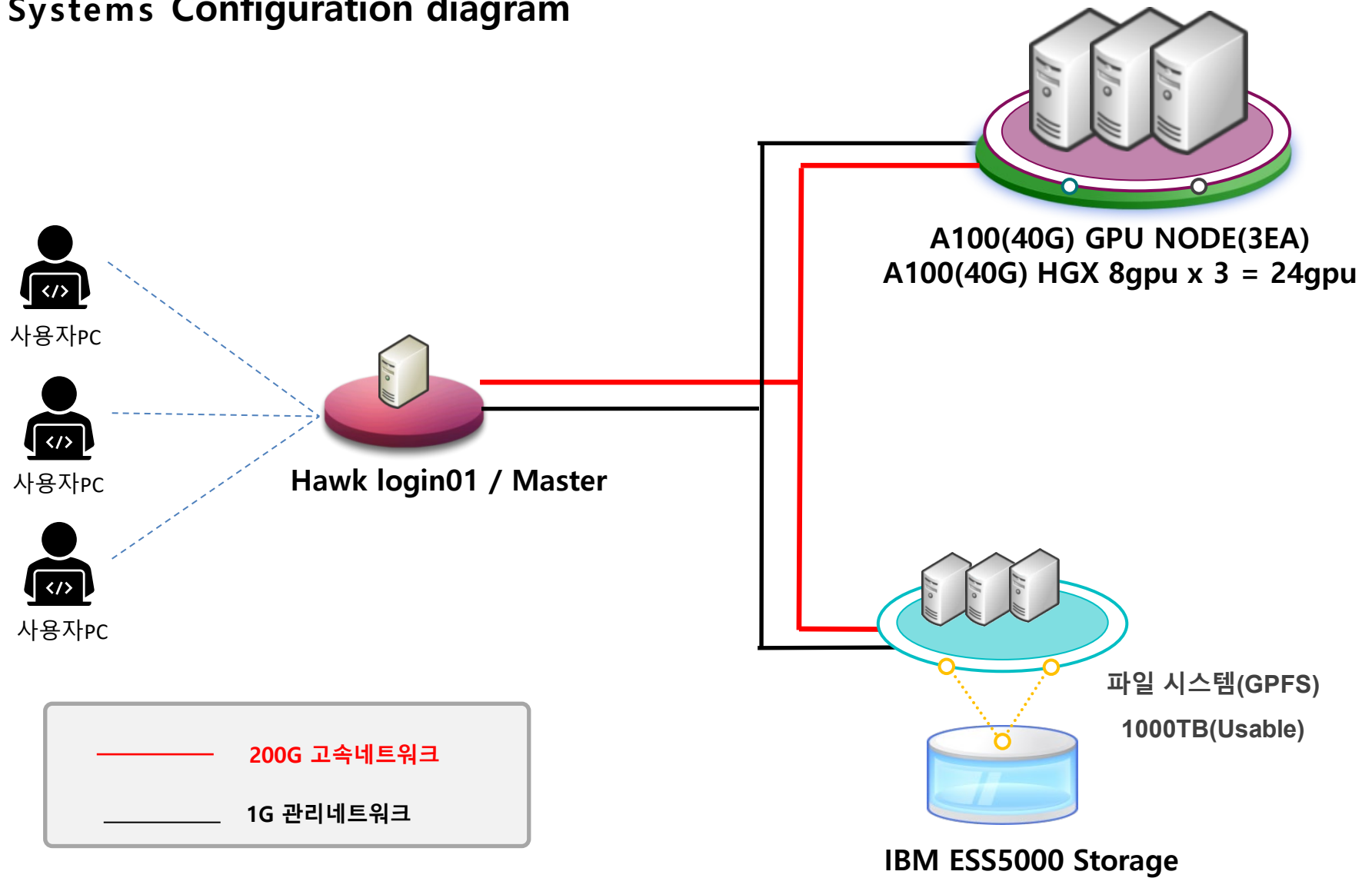
A. Configuration Overview

UNIST Supercomputing Center(이하 USC)에서는 다양한 연구 지원을 위해 High Performance Computing(이하 HPC) system 과 병렬 파일 시스템을 지원합니다. USC에서는 HPC x86_64 Linux 클러스터로 Tachyon, Leopard, Lion, Falcon, Eagle, Dumbo이 있으며 파일 시스템으로 Lustre parallel filesystem, Spectrum Scale(GPFS)을 보유하고 있습니다.

HAWK GPGPU Cluster 시스템은 노드 당 A100(40G) HGX(Nvlink) 8개의 GPU로 구성되어 있고, 총 노드는 3노드이며 GPU수로는 A100(40G) 24개를 보유하고 있습니다. 각 node 당 128개의 코어를 장착하고 있으며, 1024G Memory를 장착하고 있습니다. CPU architectur는 AMD EPYC 7742 이고, GPU architectur는 nVidia A100(40G) 입니다. 스토리지는 1PB의 IBM ESS5000 GPFS 병렬파일시스템을 사용하고 있으며 공유스토리지와 각 노드들간에는 초고성능 Infiniband 200Gbps네트워크로 연결되어 있습니다.

자세한 HPC system 구성은 다음 그림을 통해 확인할 수 있습니다.

B. Systems Configuration diagram



C. How to access HPC systems

Hawk GPU-HPC시스템에는 1개의 login node가 있습니다. 사용자는 Login node를 통해서 HPC 시스템에 접근할 수 있습니다. 사용자 PC와 login node 간 연결은 SSH protocol(2123번 포트) 을 통해 이루어지고, 이것을 가능하게 해 주는 터미널 프로그램(PuTTY, Secure Shell Client, Xmanager 등)을 이용하시면 됩니다.

Hostname	hlogin01
IP	10.0.7.71

※ UNIST supercomputing center에 설치된 시스템은 UNIST 외부에서 접속이 불가능합니다. (특수한 경우, 허용된 IP에 의해서만 접근 허용)

D. Disk quota

Hawk GPU-HPC의 스토리지는 GPFS를 통해 사용자의 홈디렉토리가 전체 노드에서 공유됩니다. GPU-HPC 시스템에는 스크래치 디렉토리가 없습니다. 할당받은 홈디렉토리 공간에서 자유롭게 작업을 하시면 됩니다.

디렉토리 이름	User 할당 용량
/home/사용자ID	1 TB

O Disk Quota: 계정당 1TB Hardlimit

E. Hardware Spec (HPC Dumbo Cluster)

i. System Overview

1) GPU Nodes

A) A100 GPU Nodes Spec

구분	Model	CPU	core	Socket	GHz	Memory	DISK	Node #
GPU 노드	NF5488A5	AMD EPYC 7742	64	2	2.25	1024G (32G*32) 3200MHz	1.92TB SSD x2	3
		NVIDIA Tesla A100 GPU 8EA						

구분	Model	용량 (Usable)	DISK	DISK 수량	etc
스토리지	ESS5000	1000TB	NL- SAS 7.2K 16TB	90	hot spare disk 2개를 별도 구성

F. Software

i. Compilers

Compilers	Version
GNU(GCC)	GCC 4.8.5(Linux CentOS 7.8 Default)
Intel compiler	Intel parallel studio 2020(2020.4.9)

ii. Libraries

Classification	Libraries	Libraries built with follow packages.
Parallel libraries	OpenMPI 3.1.6	GCC 4.8.5(Linux CentOS 7.8 Default)
	Mvapich 2.3.4	GCC 4.8.5(Linux CentOS 7.8 Default)
	Intel-impi 2020	Intel Parallel Studio 2020(2020.4.9)
Mathematical libraries	LAPACK 3.9.0	GCC 4.8.5(Linux CentOS 7.8 Default)
	NETCDF 4.6.1	GCC 4.8.5(Linux CentOS 7.8 Default)
	HDF4 4.2.13	GCC 4.8.5(Linux CentOS 7.8 Default)
	HDF5 1.10.2	GCC 4.8.5(Linux CentOS 7.8 Default)
	Intel MKL	Intel Parallel Studio 2020(2020.4.9)
GPU compiler library	Cuda 10.2, nvcc	GCC 4.8.5(Linux CentOS 7.8 Default)
AI Package	Miniconda	
	Jupyterhub	
Container	Nvidia-docker	

2. Slurm Schedule(for Job Control)

A. Overview

USC에서는 사용자가 code를 실행하기 위해서 queue system을 사용해야 합니다. 저희 USC에서는 사용자의 효과적인 사용 및 편의를 위해 SLURM(Simple Linux Utility for Resource Management)를 사용하여 job을 관리 합니다.

B. SLURM commands

Commands	Example	Description
sbatch [option...] script	\$sbatch job_script_file_name	submit a job.
squeue	\$squeue	show job status oneself
	\$squeue -u abc	show job status for 'abc' user
	\$squeue -j 12345	show job status for '12345' JobID
sinfo	\$sinfo	show computing node & partition status
	\$sinfo -N -l	show computing node & partition status detail
scancel	\$scancel [job_id]	Cancel a job using its ID
	\$scancel [user_name]	Cancel all jobs from a user

C. Slurm partition information

Partition name	wall clock time	Available cores	Available GPU	User Nodes	Memory CPU cores	Use Account	Remarks
a100_40g	12 hours	Up to group quota	Up to group quota	A100(40G) HGX GPU nodes Hawk[01-03]	Per 1GPU 128GB mem 16 cpu cores	ngpu account	If the wall time is not set or is set to exceed the specified time, the Job will not work. Up to group quota (8GPU)
a100_40g	300 hours	Up to group quota	Up to group quota	A100(40G) HGX GPU nodes Hawk[01-03]	Per 1GPU 128GB mem 16 cpu cores	Lgpu account	If the wall time is not set or is set to exceed the specified time, the Job will not work. Up to group quota (2GPU)

D. sinfo (show computing node & partition status)

\$sinfo

```

PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
gpu_k80    up     infinite   12     idle dumb0[060-071]
cpu_haswell* up     infinite   27     idle dumb0[072-079,082-100]
debug      up     3-00:00:00  1     idle dumb0080

```

\$sinfo -N -l

NODELIST	NODES	PARTITION	STATE	CPUS	S:C:T	MEMORY	TMP_DISK	WEIGHT	AVAIL_FE	REASON
dumb0060	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0061	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0062	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0063	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0064	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0065	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0066	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0067	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0068	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0069	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0070	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0071	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0072	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0073	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0074	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0075	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0076	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0077	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0078	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0079	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0080	1	debug	idle	16	2:8:1	30000	0	1	Tesla K8	none
dumb0082	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0083	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0084	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0085	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0086	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0087	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0088	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0089	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0090	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0091	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0092	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0093	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0094	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0095	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0096	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0097	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0098	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0099	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0100	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none

E. sbatch (submit a job)

```
$sbatch ./job_script.sh
```

[작업 스크립트 예제, GPU MPI 기준]

```
#!/bin/sh
```

```
#SBATCH -J thin_lfts      ## Job Name
```

```
#SBATCH -p a100_40g      ## Partition name
```

```
#SBATCH -N 1              ## node count 총 필요한 컴퓨팅 노드 수
```

```
#SBATCH -n 16             ## 1GPU 당 16core, 만약 1GPU 당 16core 이상 할당 시 무한 Holding
```

```
#SBATCH -o %x.o%j        ## filename of stdout, stdout 파일 명(.o)
```

```
#SBATCH -e %x.e%j        ## filename of stderr, stderr 파일 명(.e)
```

```
#SBATCH --time 2:00:00    ## 최대 작업 시간(Wall Time Clock Limit) 시:분:초
```

```
#SBATCH --gres=gpu:1      ## number of GPU(s) per node
```

```
source /etc/profile.d/modules.sh
```

```
module load intel/20.4.9 mip/impi-20.4.9 ##필요한 Module 로드
```

```
mpirun ./lfts_cuda_single.out inputs    ##해석 실행 코드 입력
```

[작업스크립트 작성 안내]

slurm에게 전달해야 할 정보는 작업스크립트 내에 #SBATCH 지시자를 붙여 전달합니다.

1) Job Name, 작업 이름

- 작업 이름을 명시하지 않으면 slurm이 임의로 부여
- 작업 이름을 test 로 지정할 경우 "#SBATCH --job-name=test" 또는 "#SBATCH -J test" 와 같이 사용합니다.

2) Partition Name, 작업 파티션 이름

- 작업 수행할 파티션을 지정하는 것이며, 사용가능한 파티션 이름은 sinfo 명령어로 확인이 가능
- 파티션을 gpu_k80으로 사용할 경우, "#SBATCH --partition=gpu_k80" 또는 "-p gpu_k80" 와 같이 사용합니다.

3) 필요한 자원의 양

- 작업 수행에 필요한 총 프로세스 수는 "-n" 옵션으로, 작업 수행에 필요한 컴퓨팅 노드 개수는 "-N"으로 정의합니다.
※ -n 프로세스 수는 GPU 개수와 연동되며 1GPU 당 16core로 작성해야함, 예를 들어 4GPU 사용시에는 64core 이하로 할당 가능
이 규칙에 맞지 않을 시 Job 제출이 되지 않거나 제출되더라도 계산이 시작되지 않음(무한 대기)

4) output

- Standard output을 저장할 파일명을 정의합니다.
- 파일명을 test.out으로 사용할 경우, "#SBATCH --output=test.out" 또는 "#SBATCH -o test.out" 과 같이 사용합니다.

5) error

- Standard error 를 저장할 파일명을 정의합니다.
- 파일명을 test.err으로 사용할 경우, "#SBATCH --error=test.err" 또는 "#SBATCH -e test.err" 와 같이 사용합니다.

6) Wall Time Clock Limit

- 예상되는 작업 소요 시간을 의미하며, 실제 예상되는 작업 소요 시간보다 약간 더 길게 설정해 주는 것이 안전합니다.
- 만약 해당 작업이 설정해 둔 시간내에 종료 되지 않을 경우, Wall Time Clock Limit 시간을 초과하는 시점에서 slurm이 해당 작업을 강제로 종료함.
- 이 키워드는 작업 제출 스크립트 파일에 반드시 지정되어야 하며, 초단위 또는 시:분:초 형식으로 지정할 수 있습니다.
- Wall Time Clock Limit을 1시간을 지정할 경우, "#SBATCH --time=01:00:00" 또는 "#SBATCH -t 01:00:00" 과 같이 사용합니다.

7) GPU 사용

- 1노드 당 HGX 8GPU로 구성되어 있으며, 2의 배수 GPU 계산이 효율적임, 2, 4, 8 GPU 계산을 추천함

F. `squeue` (작업목록 정보 조회)

```
[root@dumbo slurm_test]# squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
92311	debug	test	moasys	R	0:08	2	dumbo088,dumbo089
88915	gpu	GPU_test	moasys	PD	0:00	1	(Resources)
91716	cpu	hello_te	moasys	R	0:00	2	dumbo093,dumbo094
91791	cpu	hello_te	moasys	R	0:00	2	dumbo090,dumbo087
91792	cpu	hello_te	moasys	CG	0:00	2	dumbo090,dumbo091

1) `dumbo` 노드에서 `squeue` 명령어 실행

`squeue - ST`

R - 작업이 컴퓨팅 노드에서 실행 중

PD - 작업이 컴퓨팅 노드에서 대기 중

CG - 작업 완료

`squeue - REASON`

(Resources) - 작업이 컴퓨팅 노드를 사용할 수 있을 때까지 기다리고 대기

(Priority) - 우선 순위가 더 높은 작업이 컴퓨팅 노드를 기다리고 있습니다.

(ReqNodeNotAvail) - 작업에서 요청한 컴퓨팅 노드는 다음과 같은 다양한 이유로 사용할 수 없음

including: cluster downtime, nodes offline, temporary scheduling backlog

G. scontrol (제출된 작업 상세 조회)

scontrol 명령어를 이용하면 제출된 작업의 상세내역을 조회할 수 있습니다.

\$scontrol show job [작업ID]

```
[root@dumbo slurm_test]$ scontrol show job 371
JobId=371 JobName=gpu_mpi
  UserId=moasys(1000) GroupId=moasys(1000) MCS_label=N/A
  Priority=4294901740 Nice=0 Account=g_group QOS=normal
  JobState=FAILED Reason=NonZeroExitCode Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=2:0
  RunTime=00:00:00 TimeLimit=01:00:00 TimeMin=N/A
  SubmitTime=2021-04-02T13:21:30 EligibleTime=2021-04-02T13:21:30
  AccrueTime=2021-04-02T13:21:30
  StartTime=2021-04-02T13:21:30 EndTime=2021-04-02T13:21:30 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2021-04-02T13:21:30
  Partition=gpu_k80 AllocNode:Sid=dumbo:11244
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=dumbo[060-061]
  BatchHost=dumbo060
  NumNodes=2 NumCPUs=8 NumTasks=8 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  TRES=cpu=8,energy=2,node=2,billing=8,gres/gpu=4
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=0 MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=USER Contiguous=0 Licenses=(null) Network=(null)
  Command=/gpfs/home/moasys/slurm_test/gpu_mpi.sh
  WorkDir=/gpfs/home/moasys/slurm_test
  StdErr=/gpfs/home/moasys/slurm_test/gpu_mpi_371.err
  StdIn=/dev/null
  StdOut=/gpfs/home/moasys/slurm_test/gpu_mpi_371.out
  Power=
  TresPerNode=gpu:2
  MailUser=moasys MailType=NONE
```

F. scancel (cancel a job)

\$scancel [작업 ID]

3. Support Apps user-guide

A. Environment Settings

i. Overview

설치된 모든 software의 사용 편의를 위해 Environment Module이 설정되어 있습니다. 사용자들은 프로그램 동작을 위해 HPC에 서 환경 설정 자주 사용하는데 기존의 방법대로 설정하자면 실행파일, 경로, 라이브러리 경로 등을 추가해줘야 하는데 UNIX를 자주 쓰던 사용자는 익숙할 수 있지만 처음 사용하는 사용자에게는 번거로운 작업입니다. 때문에 저희 HPC system에서는 이와 같은 수고스러움을 방지하고자 module system을 사용하고 있습니다. Module file을 생성해 놓아서 필요한 module을 "load"하거 나 "unload"하면 됩니다. 추가로 사용자가 필요로 할 시 ".file"을 수정하여 개별 module file을 변경할 수 있습니다.

module 을 사용하게 되면 경로, 라이브러리 경로, MAN경로 등을 업데이트 할 때 쉽게 변경할 수 있습니다.

HPC system에서 계정이 생성되었을 때 ".file"을 사용하여 user environment를 구성 및 변경해야 되지만(default는 bash shell), module을 사용하면 좀더 쉽게 user environment를 설정할 수 있습니다.

ii. Module commands

아래는 module에 대한 command입니다.

```
$ module help
```

"help" 옵션은 아래와 같이 출력됩니다. (help를 입력하지 않아도 동일하게 출력됩니다.)

```
Available SubCommands and Args:
+ add|load      modulefile [modulefile ...]
+ rm|unload     modulefile [modulefile ...]
+ switch|swap    [modulefile1] modulefile2
```

```
+ display|show      modulefile [modulefile ...]
+ avail              [modulefile [modulefile ...]]
+ purge
+ list
```

\$ module list

현재 modulefile 을 출력합니다

\$ module avail

사용할 수 있는 모든 modulefiled 을 출력합니다.

\$ module purge

현재 load 된 modulefile 을 모두 unload 합니다.

\$ module load *modulefile*

"*modulefile*"을 load 합니다.

\$ module unload *modulefile*

"*modulefile*"을 unload 합니다.

\$ module switch modulefile_old modulefile_new

"*modulefile_old*"을 "*modulefile_new*"로 변경 합니다.

\$ module show *modulefile*

"*modulefile*"을 PATH, LD_LIBRARY_PATH, MANPATH, etc 로 변경하는 방법을 출력합니다.

B. Conda Environment settings(using miniconda)

i. Overview

GPU기반의 AI Framework를 사용하기 위해서는 CONDA 환경을 이용하셔야 합니다. 사용자들은 기본적으로 설치된 module 에서 AI Framework를 사용할 수 있고, 개인 폴더에서 자신만의 환경을 구성할 수 있습니다.

ii. Module commands

아래는 module에서 conda 환경을 로드하는 방법입니다.

```
$ module list
```

```
----- /apps/Modules/modulefiles/conda_packages -----  
Conda/pytorch      conda/tensorflow      conda/tensorflow-gpu
```

```
$ module load conda/tensorflow-gpu
```

설치되어있는 모듈은 conda-forge에서 현재 설치된 가장 안정적인 버전으로 설치된 버전입니다. 다만 이 경우 최신 버전의 framework를 사용하지 못함으로 개인적으로 자신이 필요한 파일을 설치하시면 됩니다.

Module의 경우는 python에서 import 되어있는 환경을 그대로 사용이 가능함으로 conda load와 같이 환경이 변경되지는 않습니다.

Module에서 로드한 환경에서의 python은 필요 버전에 맞게 설치됨으로 각 버전에 설치된 conda 정보를 확인하려면 아래의 명령을 수행합니다.

```
$ CONDA_HOME=/apps/applications/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh  
$ conda env list
```

iii. 사용자 폴더에서의 conda 환경 사용

사용자 폴더에서 필요한 conda 환경을 로드하기 위해서는 아래와 같은 명령을 수행한다.

```
$ CONDA_HOME=/apps/applications/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh
```

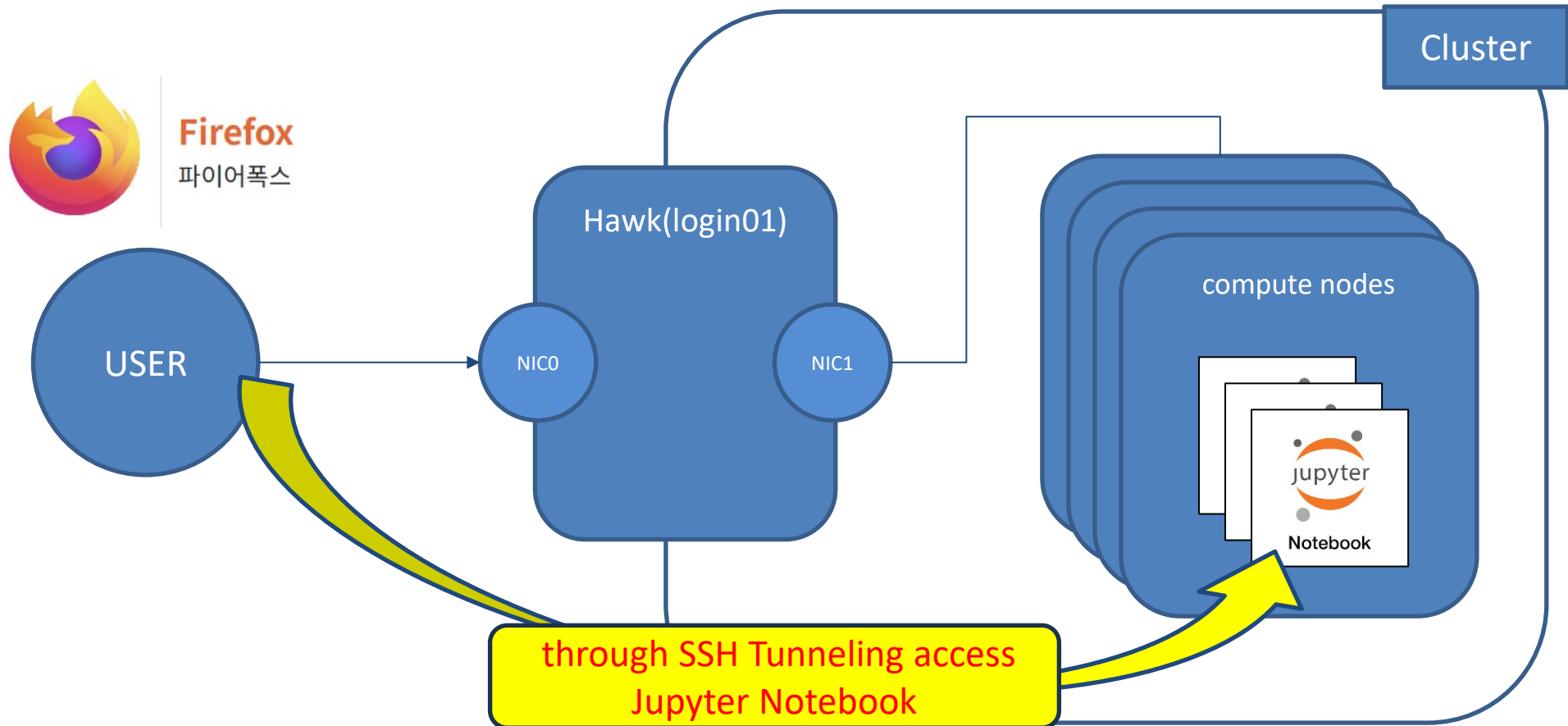
Conda 환경설정을 수행한 후 각 사용자 홈폴더에 conda 환경을 구성한다.

```
$ CONDA_HOME=/apps/applications/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh  
$ conda create -n tf_2.2_gpu  
Collecting package metadata (current_repodata.json): done  
Solving environment: done  
  
## Package Plan ##  
  
environment location: /home/moasys/.conda/envs/tf_2.2_gpu  
  
Proceed ([y]/n)? y  
  
Preparing transaction: done  
Verifying transaction: done  
Executing transaction: done  
#  
# To activate this environment, use  
#  
# $ conda activate tf_2.2_gpu  
#  
# To deactivate an active environment, use  
#  
# $ conda deactivate
```

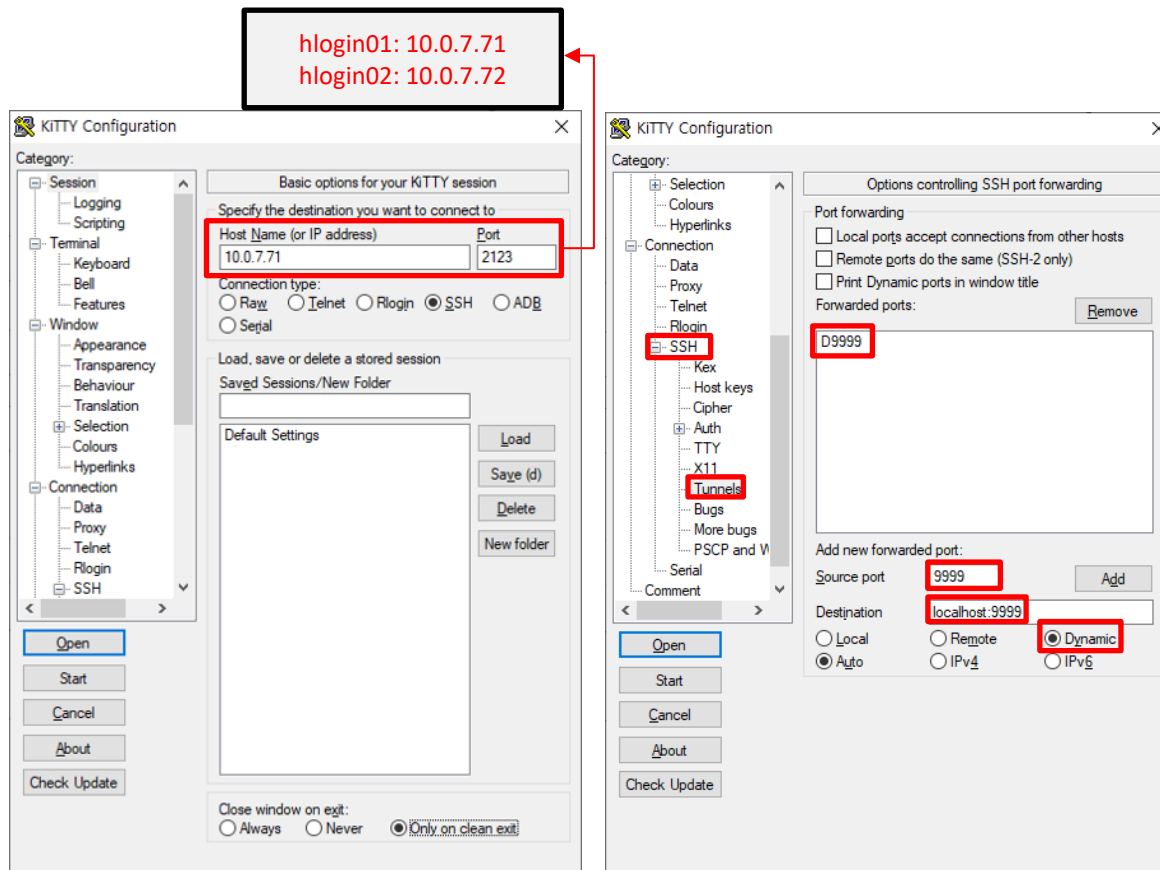
C. Jupyter Notebook Settings

i. SSH tunneling 설정 개요

- SSH tunneling 을 통해 User PC Web 브라우저 에서 Jupyter Notebook을 띄워
- UNIST 슈퍼컴퓨팅센터 시스템에 접속해서 사용 가능하도록 설정 방법 안내



ii. SSH tunneling 설정 (putty 기준 설정)

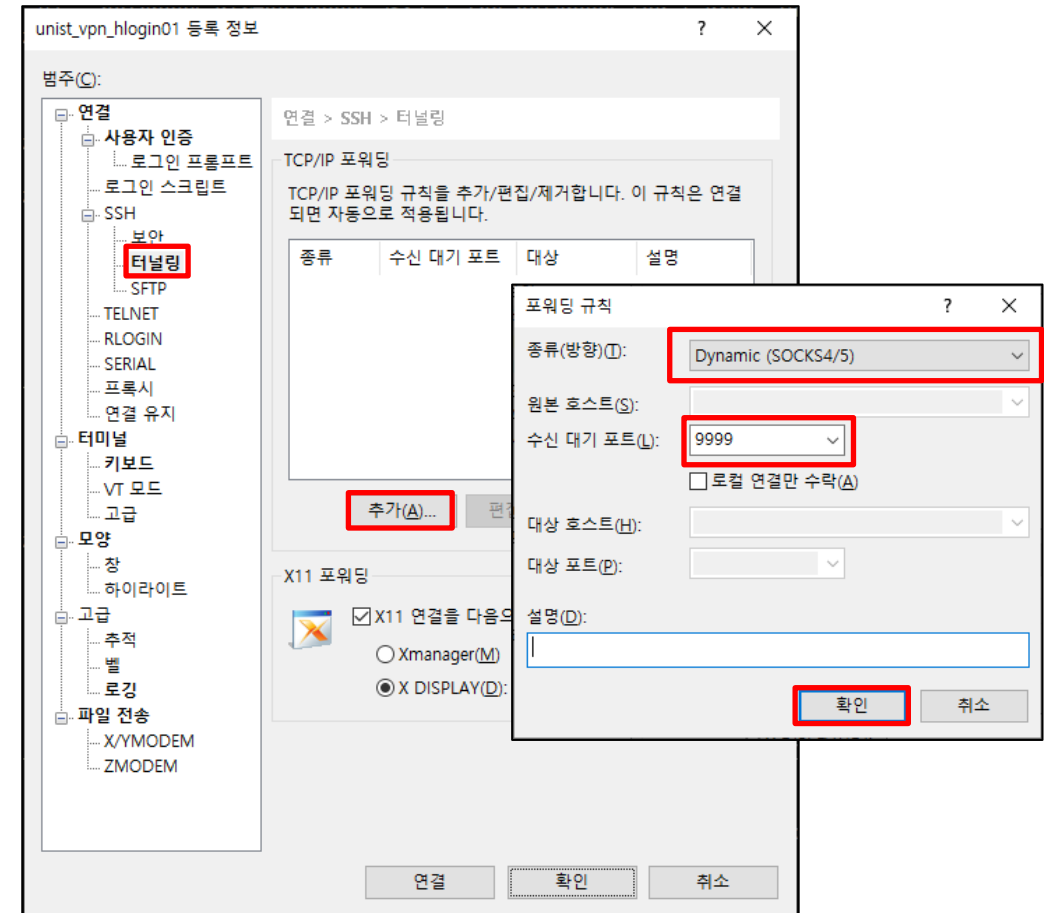
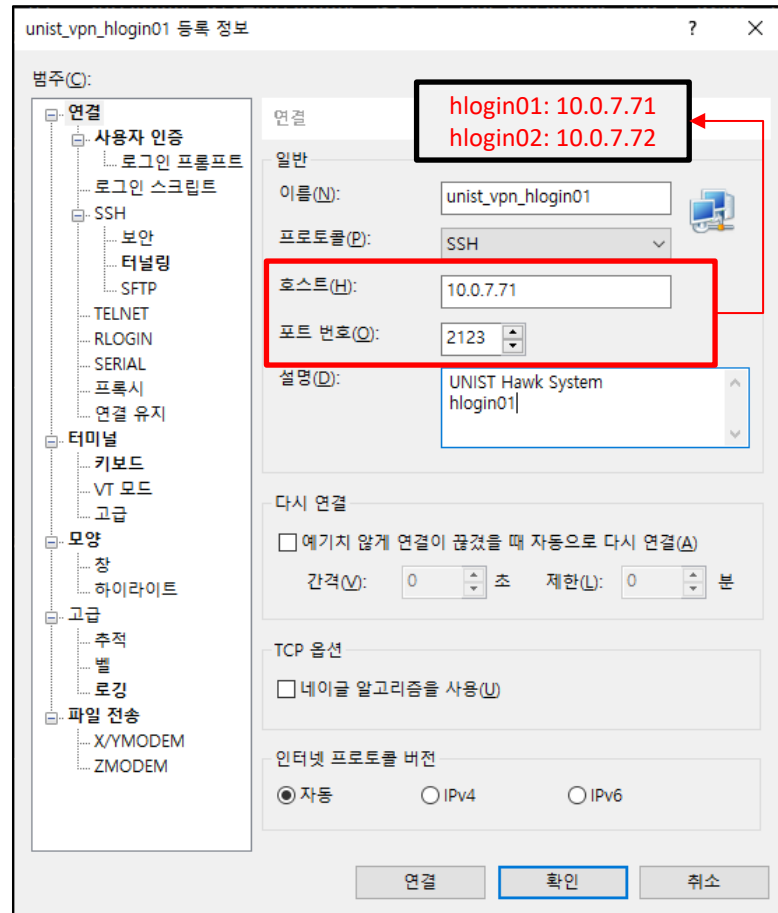


- 좌측 메뉴 SSH → Tunnels
- Source port: 9999
- Destination: localhost:9999
- Dynamic 선택
- Add 버튼으로 항목 추가

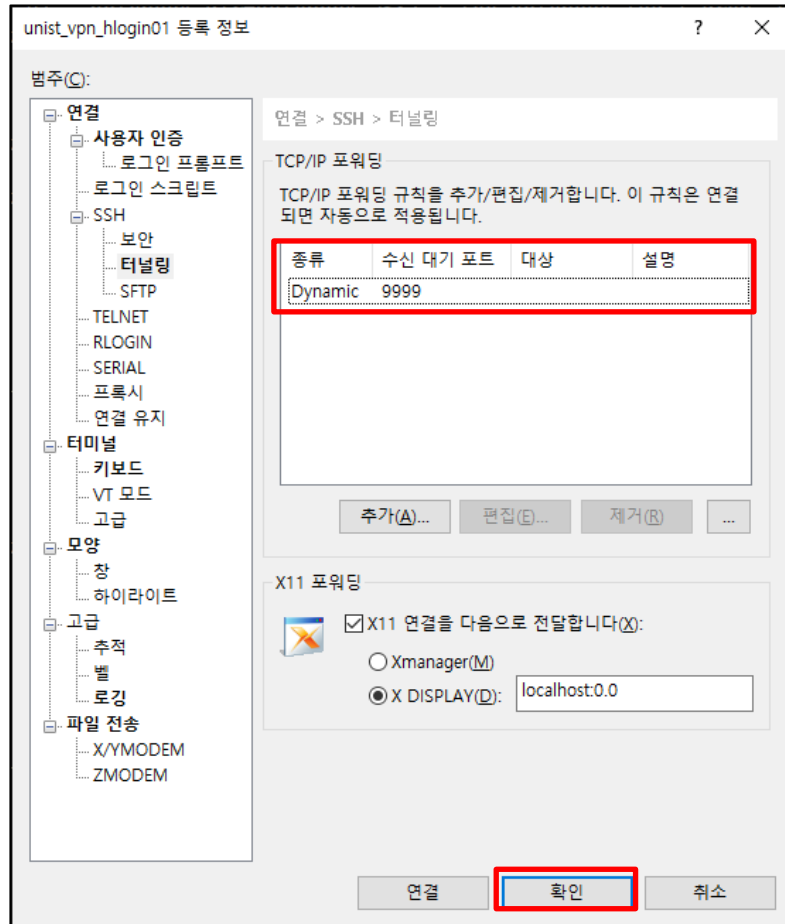
예시 이미지는 kitty나 putty와 대체로 동일한 구성인 점 참고

- SSH 터널링 설정 완료
- 설정 완료된 프로파일로 서버에 접속 후 로그인하면 SSH 터널링 기능이 활성화됨

iii. SSH tunneling 설정 (Xshell의 경우) 1/2



iii. SSH tunneling 설정 (Xshell의 경우) 2/2

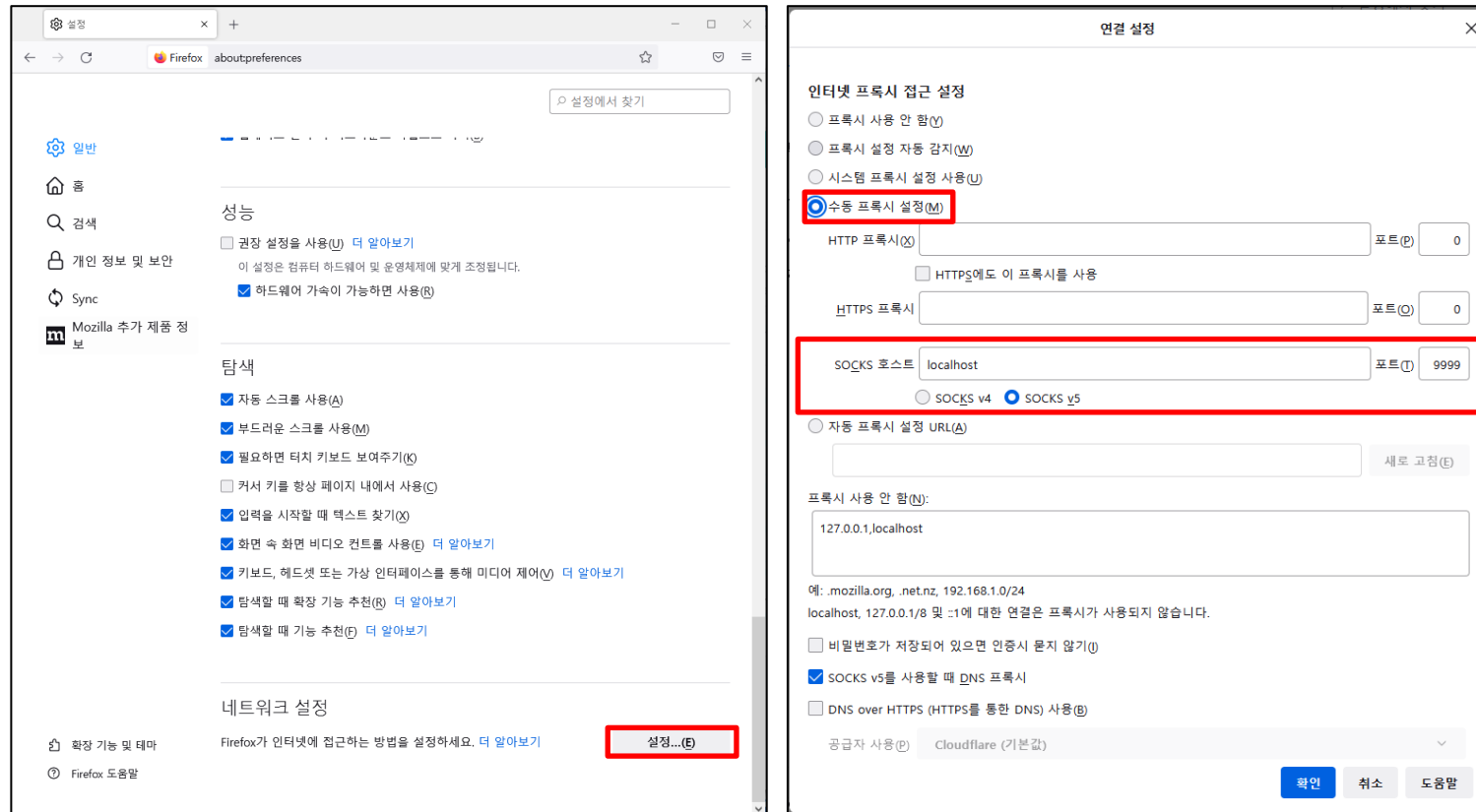


- SSH 터널링 설정 완료
- 설정 완료된 프로파일로 서버에 접속 후 로그인하면 SSH 터널링 기능이 활성화됨

iv. 내PC 쪽(Windows/Mac) 브라우저 프록시 설정(Firefox 기준 설명)



Firefox
파이어폭스

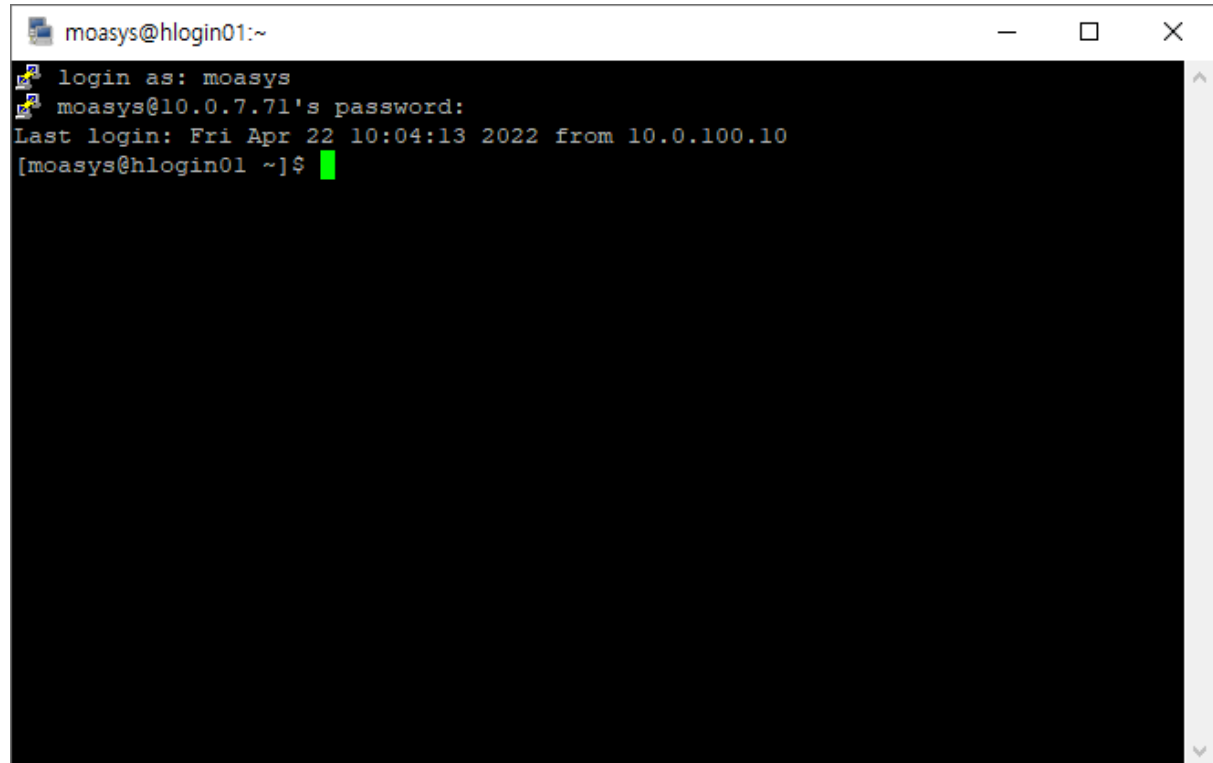
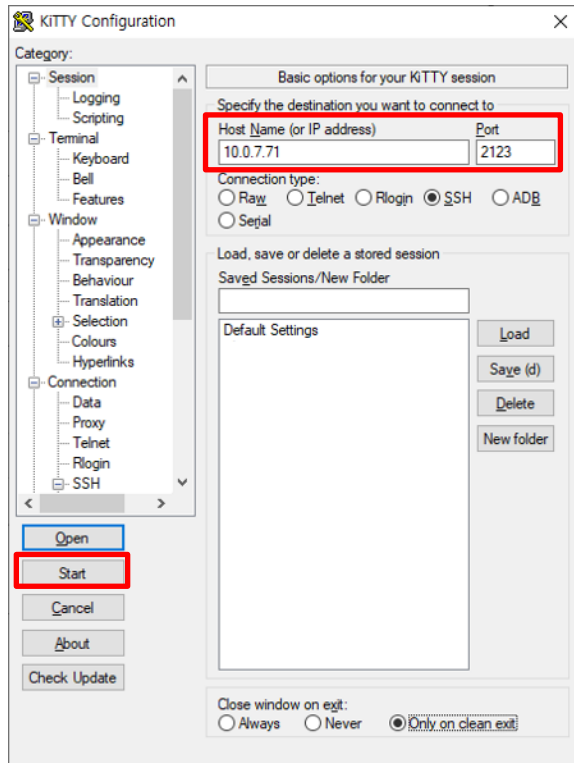


- 수동 프록시 설정 → SOCKS 호스트 설정
- SSH 터널링 설정에서 지정한 포트 입력
→ localhost:9999

v. 터널링 설정 후 UNIST 슈퍼컴퓨터(리눅스) 접속 후 설정 1/3

Jupyter Notebook 실행 및 접속 방법

- SSH Tunneling 설정된 클라이언트로 로그인 서버(hlogin01(10.0.7.71) 접속
로그인 성공 시 SSH 터널링 활성화됨



XSHELL
PLUS

v. 터널링 설정 후 UNIST 슈퍼컴퓨터(리눅스) 접속 후 설정 2/3

Jupyter Notebook 실행 및 접속 방법

- Jupyter Notebook script Example - myjupyter.sh 다음페이지에서 사용 방법 안내

```
#!/bin/sh
#SBATCH - J myjupyter
#SBATCH - p gpu
#SBATCH - N 1
#SBATCH - n 16
#SBATCH - o %x.%j
#SBATCH - e %x.%j
#SBATCH -- time 12:00:00
#SBATCH -- gres=gpu:1
#SBATCH -- comment etc
#SBATCH -- nodelist=hawk03
```

Output 파일명: <JobName>.<JobID>
본 스크립트에서는 myjupyter.<JobID>

활성화할 jupyter notebook conda
환경 이름 지정

```
source /apps/applications/miniconda3/bin/activate notebook
```

```
jupyter notebook --no-browser --ip=$(hostname -i) --port=5$(id -u) --notebook-dir=$HOME/notebook
```

jupyter notebook 작
업 경로 지정

v. 터널링 설정 후 UNIST 슈퍼컴퓨터(리눅스) 접속 후 설정 3/3

Jupyter Notebook 실행 및 접속 방법

- 실행된 터미널에서 이전 단계에서 작성한 batch job script 제출

```
$ sbatch myjupyter.sh
```

```
Submitted batch job 1234
```

앞 페이지 스크립트 예제 참조

- batch job이 정상적으로 실행되었는지 확인 후 접속 URL 복사

```
$ squeue -u $USER
```

```
$ cat myjupyter.1234
```

```
...
```

```
...
```

To access the notebook, open this file in a browser:

```
file:///gpfs/home/moasys/.local/share/jupyter/runtime/nbserver-32164-open.html
```

Or copy and paste one of these URLs:

```
http://172.30.0.13:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6
```

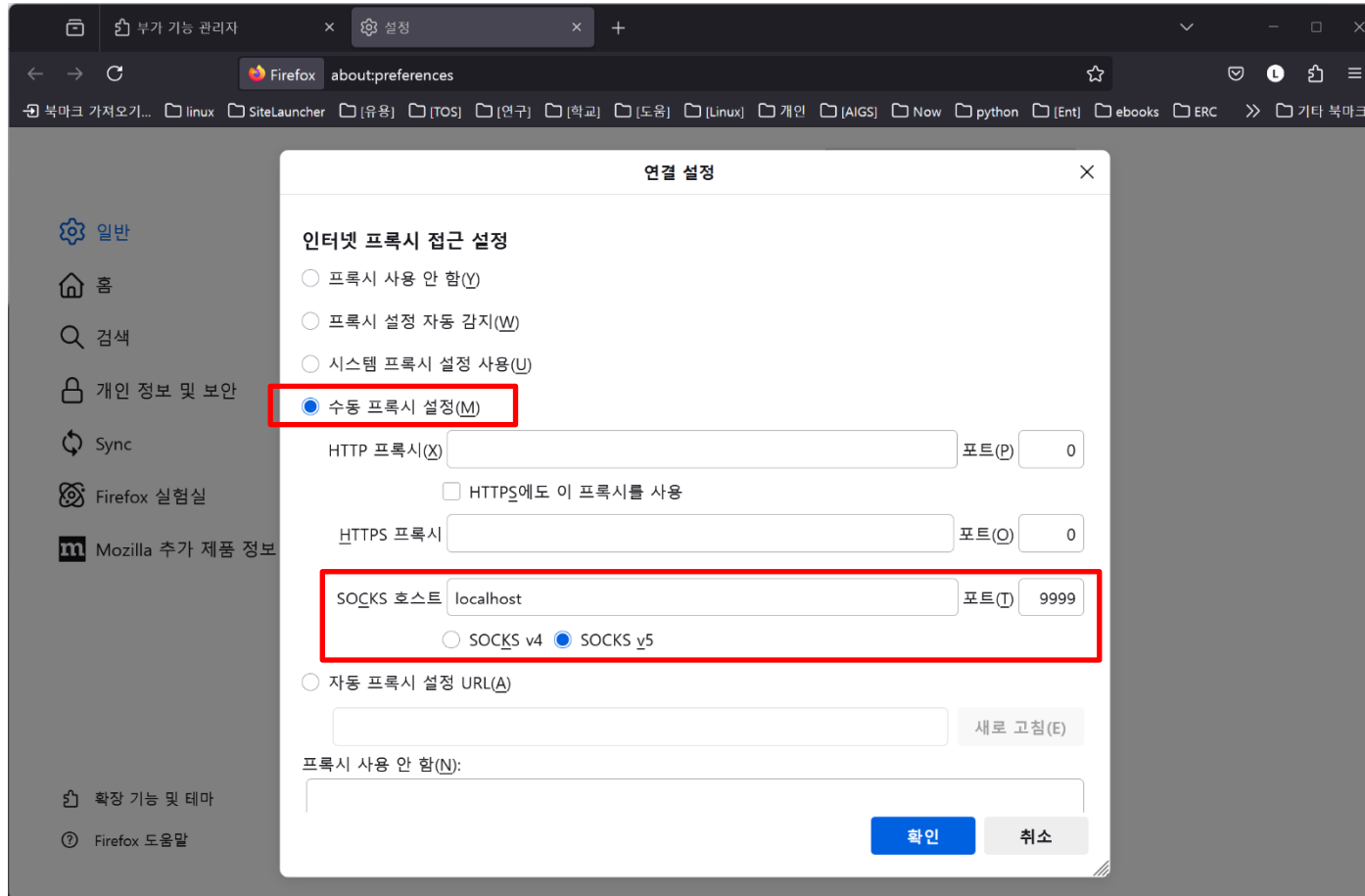
```
or http://127.0.0.1:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6
```

Or copy and paste...바로 아래줄의 URL 및 토큰 복사.
127.0.0.1은 loopback IP이므로 원격 웹 브라우저 접속 시 사용 불가

vi. 터널링 설정 후 내PC에서 브라우저 접속(Firefox 기준 설명) 1/2

Jupyter Notebook 실행 및 접속 방법

- Firefox에서 SOCKS v5 프록시 설정
 - ‘설정’ -> ‘일반’ -> ‘네트워크 설정의 ‘설정...(E)’

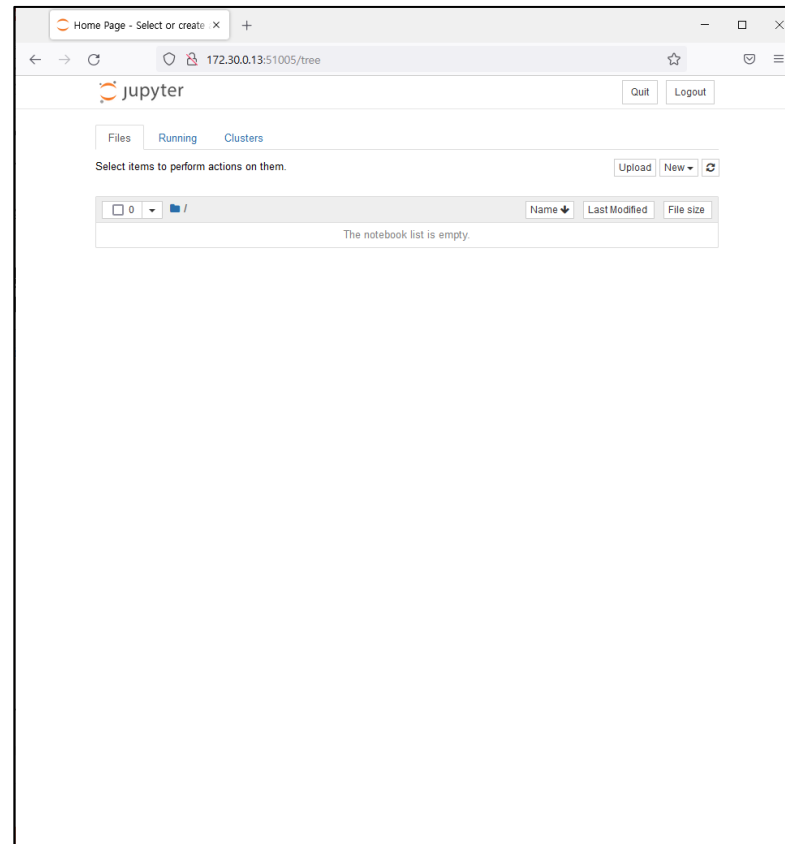
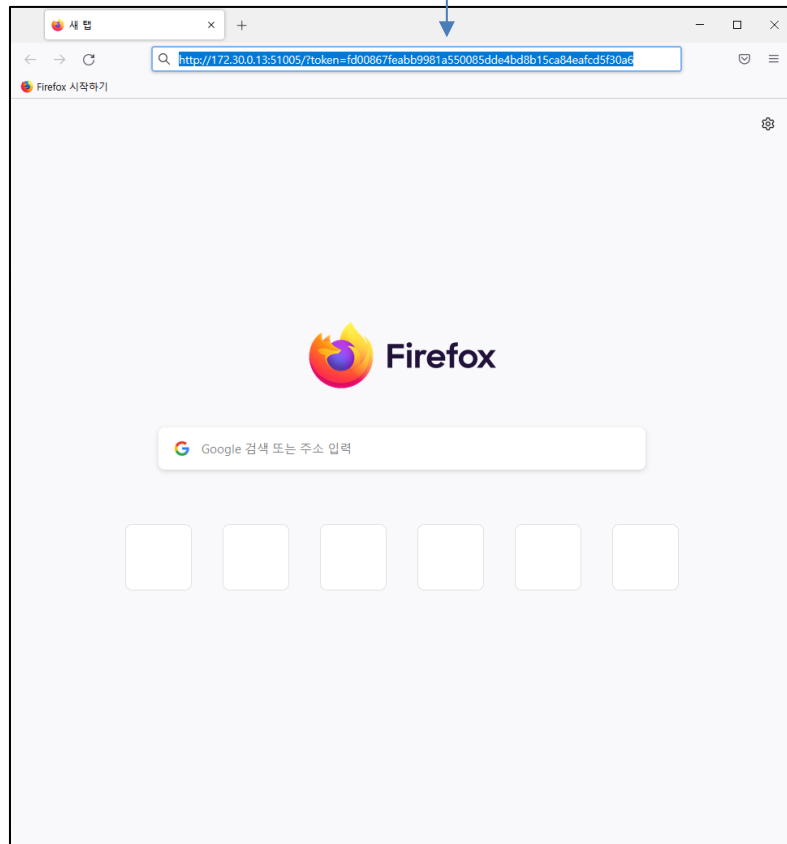


vi. 터널링 설정 후 내PC에서 브라우저 접속(Firefox 기준 설명) 2/2

Jupyter Notebook 실행 및 접속 방법

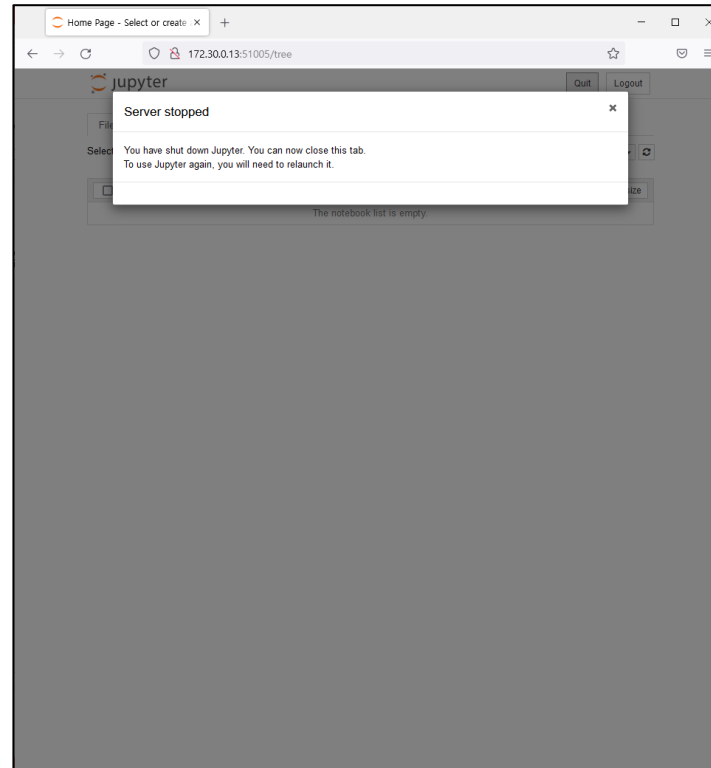
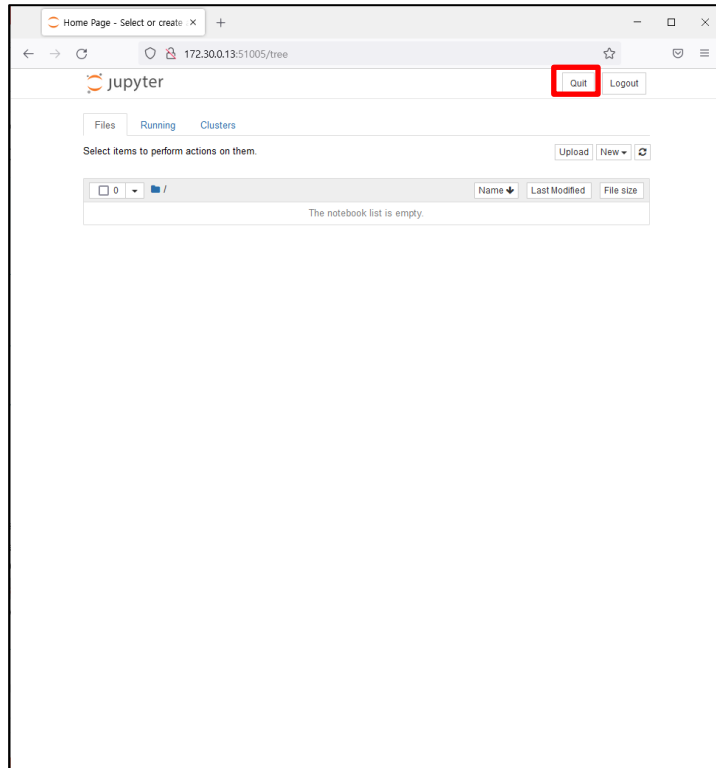
- SOCKS v5 프록시가 설정된 브라우저 실행 후 주소란에 이전 단계에서 복사한 Jupyter Notebook URL 및 토큰 붙여넣어 접속

<http://172.30.0.13:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6>



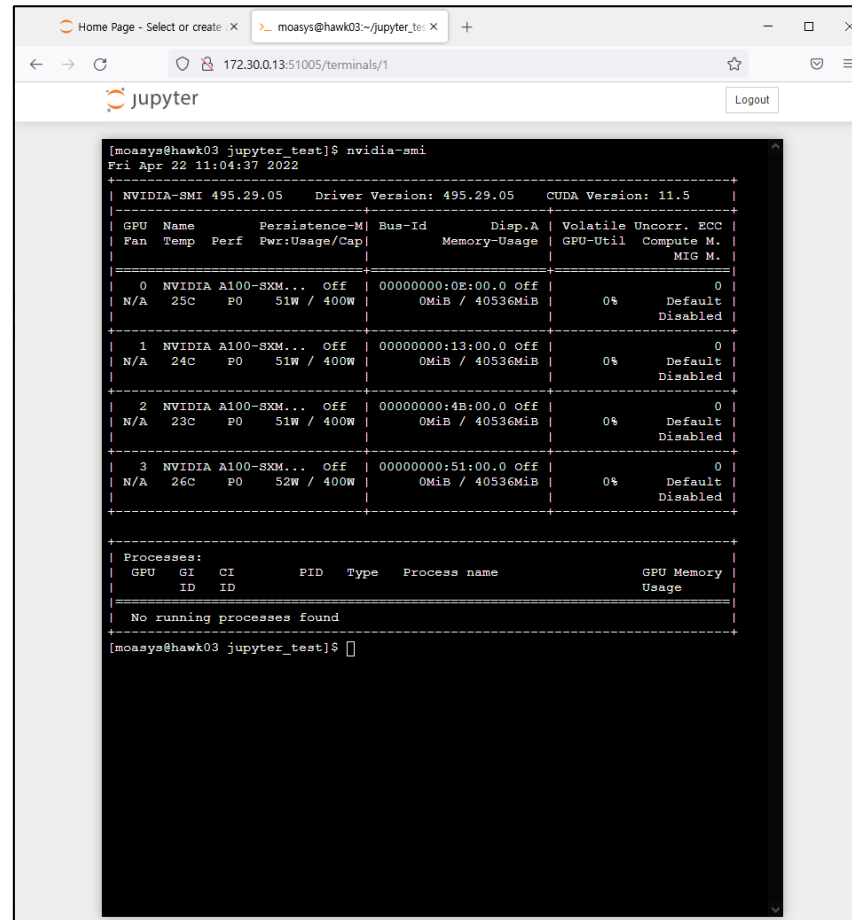
vii. Jupyter Notebook 환경 확인 및 종료 1/2

- 사용이 끝난 경우 반드시 **Quit** 버튼으로 명시적으로 종료
- 명시적으로 종료하지 않고 웹 브라우저를 닫을 경우, Jupyter Notebook이 종료되지 않고 계산노드에서 실행 상태로 남아 시스템 자원 지속 점유



vii. Jupyter Notebook 환경 확인 및 종료 2/2

- 터미널에서 Jupyter Notebook을 실행중인 batch job 자체를 종료 처리하는 방법도 있으나 데이터 유실의 위험이 존재하는 점 유의
- (기타사항) **myjupyter.sh** 스크립트를 통해 (스케줄러)할당받은 자원 한도 내에서만 사용 가능함(GPU 리소스)



```
[moasys@hawk03 jupyter_test]$ nvidia-smi
Fri Apr 22 11:04:37 2022

+-----+
| NVIDIA-SMI 495.29.05      Driver Version: 495.29.05      CUDA Version: 11.5      |
+-----+-----+
| GPU   Name                               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan    Temp   Perf          Pwr:Usage/Cap|  Memory-Usage | GPU-Util  Compute M. |
|===============================================================================|
| 0      NVIDIA A100-SXM4-80GB             Off          | 00000000:0E:00:00 Off |    0MiB / 40536MiB | 0%      Default |
| N/A    25C    P0           51W / 400W    | 0MiB / 40536MiB | 0%      Default |
|                               Disabled |
+-----+-----+
| 1      NVIDIA A100-SXM4-80GB             Off          | 00000000:13:00:00 Off |    0MiB / 40536MiB | 0%      Default |
| N/A    24C    P0           51W / 400W    | 0MiB / 40536MiB | 0%      Default |
|                               Disabled |
+-----+-----+
| 2      NVIDIA A100-SXM4-80GB             Off          | 00000000:4B:00:00 Off |    0MiB / 40536MiB | 0%      Default |
| N/A    23C    P0           51W / 400W    | 0MiB / 40536MiB | 0%      Default |
|                               Disabled |
+-----+-----+
| 3      NVIDIA A100-SXM4-80GB             Off          | 00000000:51:00:00 Off |    0MiB / 40536MiB | 0%      Default |
| N/A    26C    P0           52W / 400W    | 0MiB / 40536MiB | 0%      Default |
|                               Disabled |
+-----+-----+

Processes:
+-----+
| GPU   GI   CI        PID   Type   Process name                      GPU Memory |
| ID    ID   ID                                  Usage    |
+-----+
| No running processes found |
+-----+

[moasys@hawk03 jupyter_test]$
```

D. Nvidia-Docker

i. Nvidia-Docker 스크립트 예제

```
#!/bin/bash
#SBATCH -J docker_job          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 16                  # 사용할 전체 프로세스 수 (CPU:GPU 16:4 비율로 할당하면 좋으나 선택사항)
#SBATCH -o $x.o%j             # 표준출력 파일 형식
#SBATCH -e %x.e%j             # 표준에러 파일 형식
#SBATCH --gres=gpu:4           # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00        # walltime 설정

module purge

CONT='Ubuntu' # 사용할 docker 이미지 이름으로 변경
MOUNT=/mnt # docker 이미지 내부의 마운트 포인트 지정

# SLURM 작업ID당 하나의 docker job만 수행 가능하도록 제한되어 있으므로
# 하나의 SLURM 작업 스크립트에서는 nvidia-docker를 한 번만 실행 가능함
nvidia-docker \
    run -v $PWD:$MOUNT -w $MOUNT \ # 호스트의 작업 제출 경로를 docker 이미지 내부에 마운트하고 작업 디렉터리로 지정
    $CONT \ # 사용할 docker 이미지 이름
    mpirun -np 4 mpi.exe # 실행문은 사용자 docker 이미지에 맞게 수정. MPI 작업일 경우 실행문에 실행할 MPI Process 수 명시
```

ii. Nvidia-Docker Interactive mode(1/2)

인터랙티브 작업 예제 (docker)

1) docker 컨테이너 환경을 이용하여 대화형 작업을 수행하시려면 SLURM 스케줄러를 통해 먼저 자원을 할당받으셔야 합니다.

```
$ salloc --partition=gpu -N=<노드 수> -n <사용할 전체 CPU 수> --gres=gpu:<노드당 GPU 수> --time=02:00:00
```

하나의 노드에서 최대 사용 가능한 자원은 CPU 16개, GPU 4개이며, 메모리의 경우 CPU 코어 수에 비례하여 할당됩니다. (코어당 2048MB)

CPU:GPU를 16:4 비율로 할당하면 좋으나, 강제사항은 아니며 작업에 필요한 정도로 할당을 진행하셔도 됩니다.

Docker 이미지 내부에서 작업 시, SLURM에서 할당받은 자원 한도 내에서만 작업수행이 가능하며, 할당받은 메모리 한도를 초과하여 작업을 수행하실 경우 시스템에 의해 강제로 종료될 수 있습니다.

또한 docker 작업은 단일 노드에서 수행하시는 것이 권장됩니다.

2) SLURM 스케줄러를 통해 할당받은 계산 노드로 진입합니다.

```
$ srun --pty bash
```

3) 실행 바이너리 혹은 Input 파일 등이 위치한 작업 디렉터리로 이동한 후, 인터랙티브 모드로 docker 이미지에 진입하여 작업을 수행하실 수 있습니다.

SLURM에서 할당받은 자원 설정이 자동으로 docker 작업에도 적용되므로 별도의 GPU 관련 설정(--ngpus)은 필요하지 않으며, --ngpus, --name 등의 파라미터 사용은 제한되는 점 참고부탁드립니다.

```
$ cd <작업 디렉터리>
$ nvidia-docker run -it -v $PWD:/mnt -w /mnt <docker 이미지 이름>
I have no name!@<컨테이너ID>:/mnt$ mpirun ./mpi.exe
.....
.....
I have no name!@<컨테이너ID>:/mnt$
```


ii. Nvidia-Docker Interactive mode(2/2)

4) 인터랙티브 모드로 docker 작업을 수행할 경우, 첫 "nvidia-docker run -it ..." 명령어로 이미지에 한번 진입한 후 다시 "nvidia-docker run -it ..." 명령어를 수행하려고 시도하면 다음과 같은 에러가 발생할 수 있습니다.

```
A container executed in the same SLURM job already exists
please remove user001_1234 (in hawk01)
```

SLURM을 통해 계산노드의 자원을 할당받은 경우, 리소스 관리를 위해 하나의 SLURM 작업에서는 하나의 docker 작업 수행만을 허용하고 있습니다.

하나의 SLURM 작업(인터랙티브 모드)에서 또다른 docker 작업을 수행하시려면 직전에 종료된 docker 작업을 먼저 삭제하셔야 합니다.

```
$ nvidia-docker rm <USER ID>_<SLURM_JOB_ID>
```

5) 호스트의 여러 디렉터리에 접근할 필요가 있다면 다음과 같이 -v 인수를 여러 번 사용하여 다수의 호스트 디렉터리를 마운트 가능합니다.

```
$ cd <작업 디렉터리>
$ nvidia-docker run -it \
-v $PWD:/mnt
-v <호스트 디렉터리1>:/<이미지 내부에 마운트할 경로1>
-v <호스트 디렉터리2>:/<이미지 내부에 마운트할 경로2>
-w /mnt <docker 이미지 이름>

I have no name!@<컨테이너ID>:/mnt$
```

4. 작업 스크립트 예제

i. 작업 스크립트 예제(요약)

1. CPUs serial 작업

```
#!/bin/sh
#SBATCH -J Serial_job
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell

./serial.exe
```

2. CPUs openmp 작업

```
#!/bin/sh
#SBATCH -J OpenMP_job
#SBATCH --time=01:00:00
#SBATCH -N 1 # number of node
#SBATCH -n 1
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell

export OMP_NUM_THREADS=10

./omp.exe
```

3. CPUs mpi 작업

```
#!/bin/sh
#SBATCH -J MPI_job
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 2 # number of node
#SBATCH -n 8 # total process

srun ./mpi.exe
```

4. CPUs hybrid 작업

```
#!/bin/sh
#SBATCH -J omp_mpi
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 1 # number of node
#SBATCH -n 2

export OMP_NUM_THREADS=10

srun ./a.out
```



작업 스크립트 예제(요약)

5. CPUs Array 작업

```
#!/bin/sh
#SBATCH -J omp_mpi
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 1                # number of node
#SBATCH -n 2
#SBATCH --array 1-10

export OMP_NUM_THREADS=10

srun ./a.out
```

6. GPU serial job

```
#!/bin/sh
#SBATCH -J gpu_k80
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p gpu
#SBATCH --gres=gpu:2 # using 2 gpus

./serial.exe
```

7. GPU openmp job

```
#!/bin/sh
#SBATCH -J gpu_omp
#SBATCH --time=01:00:00
#SBATCH -p gpu_k80
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -N 1 # number of node
#SBATCH -n 1
#SBATCH --gres=gpu:2 # using 2 gpus

export OMP_NUM_THREADS=10

./omp.exe
```

8. GPU mpi job

```
#!/bin/sh
#SBATCH -J gpu_k80
#SBATCH --time=01:00:00
#SBATCH -p gpu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -N 2 # number of node
#SBATCH -n 8 # total process
#SBATCH --gres=gpu:2 # using 2 gpus

srun ./mpi.exe
```

ii. 작업 스크립트 예제(상세)

A. 작업 스크립트 예제 (CPU)

1) CPU Serial Job

```
#!/bin/bash
#SBATCH -J cpu_Serial          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 1                   # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j              # 표준출력 파일 형식
#SBATCH -e %x.e%j              # 표준에러 파일 형식
#SBATCH --gres=gpu:0            # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00          # walltime 설정

module purge
module load intel/20.4.9

./serial.exe
```

2) CPU OpenMP Job

```
#!/bin/bash
#SBATCH -J cpu_OpenMP          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 1                   # 사용할 전체 프로세스 수
#SBATCH -c 8                   # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j              # 표준출력 파일 형식
#SBATCH -e %x.e%j              # 표준에러 파일 형식
#SBATCH --gres=gpu:0            # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00          # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load intel/20.4.9

./omp.exe
```

3) CPU MPI Job

```
#!/bin/bash
#SBATCH -J cpu_MPI           # 작업명
#SBATCH -p gpu               # 사용할 Partition(=queue) 지정
#SBATCH -N 1                 # 사용할 계산노드 수 지정
#SBATCH -n 8                 # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j           # 표준출력 파일 형식
#SBATCH -e %x.e%j           # 표준에러 파일 형식
#SBATCH --gres=gpu:0         # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00       # walltime 설정

module purge
module load intel/20.4.9 mpi/impi-20.4.9

srun ./mpi.exe
```

4) CPU Hybrid Job

```
#!/bin/bash
#SBATCH -J cpu_Hybrid          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 2                   # 사용할 전체 프로세스 수
#SBATCH -c 8                   # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j             # 표준출력 파일 형식
#SBATCH -e %x.e%j             # 표준에러 파일 형식
#SBATCH --gres=gpu:0           # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00         # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load intel/20.4.9 mpi/impi-20.4.9

srun ./hybrid.exe
```

B. 작업 스크립트 예제 (GPU)

1) GPU Serial Job

```
#!/bin/bash
#SBATCH -J gpu_Serial      # 작업명
#SBATCH -p gpu             # 사용할 Partition(=queue) 지정
#SBATCH -N 1               # 사용할 계산노드 수 지정
#SBATCH -n 1               # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j          # 표준출력 파일 형식
#SBATCH -e %x.e%j          # 표준에러 파일 형식
#SBATCH --gres=gpu:1        # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

module purge
module load gcc/4.8.5 cuda/11.4

./serial.exe
```


2) GPU OpenMP Job

```
#!/bin/bash
#SBATCH -J gpu_OpenMP          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 1                   # 사용할 전체 프로세스 수
#SBATCH -c 8                   # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j              # 표준출력 파일 형식
#SBATCH -e %x.e%j              # 표준에러 파일 형식
#SBATCH --gres=gpu:2            # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00          # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load gcc/4.8.5 cuda/11.4

./omp.exe
```

3) GPU MPI Job

```
#!/bin/bash
#SBATCH -J cpu_MPI          # 작업명
#SBATCH -p gpu              # 사용할 Partition(=queue) 지정
#SBATCH -N 1                # 사용할 계산노드 수 지정
#SBATCH -n 8                # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j           # 표준출력 파일 형식
#SBATCH -e %x.e%j           # 표준에러 파일 형식
#SBATCH --gres=gpu:8         # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

module purge
module load gcc/4.8.5 cuda/11.4 cudampi/openmpi-4.1.1

srun ./mpi.exe
```

4) GPU Hybrid Job

```
#!/bin/bash
#SBATCH -J cpu_MPI          # 작업명
#SBATCH -p gpu              # 사용할 Partition(=queue) 지정
#SBATCH -N 1                # 사용할 계산노드 수 지정
#SBATCH -n 8                # 사용할 전체 프로세스 수
#SBATCH -c 8                # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j           # 표준출력 파일 형식
#SBATCH -e %x.e%j           # 표준에러 파일 형식
#SBATCH --gres=gpu:8         # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

export OMP_NUM_THREADS=8
module purge
module load gcc/4.8.5 cuda/11.4 cudampi/openmpi-4.1.1

srun ./hybrid.exe
```

<Index >

1. HPC Resources	45	3. Support Apps user-guide	57
A. Configuration Overview	45	A. Environment Settings	57
B. Systems configuration diagram	46	i. Overview	57
C. How to access HPC systems	47	ii. Module commands	57
D. Disk quota	47	B. Conda Environment settings(using miniconda)	58
E. Hardware Spec (HPC Dumbo Cluster)	48	i. Overview	58
i. System Overview	48	ii. Module commands	58
1) GPU Nodes	48	iii. 사용자 폴더에서의 conda 환경 사용법(설정)	60
A) A100 GPU Nodes	48	C. Jupyter Notebook Settings	61
F. Software	49	i. SSH tunneling 설정 개요	61
i. Compilers	49	ii. SSH tunneling 설정 (putty 기준 설정)	62
ii. Libraries	49	iii. SSH tunneling 설정 (Xshell의 경우)	63
2. SLURM Schedule(for Job Control)	50	iv. 내PC 쪽(Windows/Mac) 브라우저 프록시 설정 (Firefox 기준 설명)	65
A. Overview	50	v. 터널링 설정 후 UNIST 슈퍼컴퓨터(리눅스) 접속 후 설정	66
B. SLURM Commands	50	vi. 터널링 설정 후 내PC에서 브라우저 접속 (Firefox 기준 설명)	69
C. SLURM partition information	51	vii. Jupyter Notebook 환경 확인 및 종료	71
D. sinfo (show computing node & partition status)	52	D. Nvidia-Docker	73
E. sbatch (submit a job)	53	i. Nvidia-Docker 스크립트 예제	73
F. squeue (작업목록 정보 조회)	55	ii. Nvidia-Docker Interactive mode	74
G. scontrol (제출된 작업 상세 조회)	56	4. 작업 스크립트 예제	76
F. scancel (cancel a job)	56	i. 작업 스크립트 예제(요약)	76
		ii. 작업 스크립트 예제(상세)	78

UNIST HPC-GPU System User Guide(Beta version)

1. HPC Resources

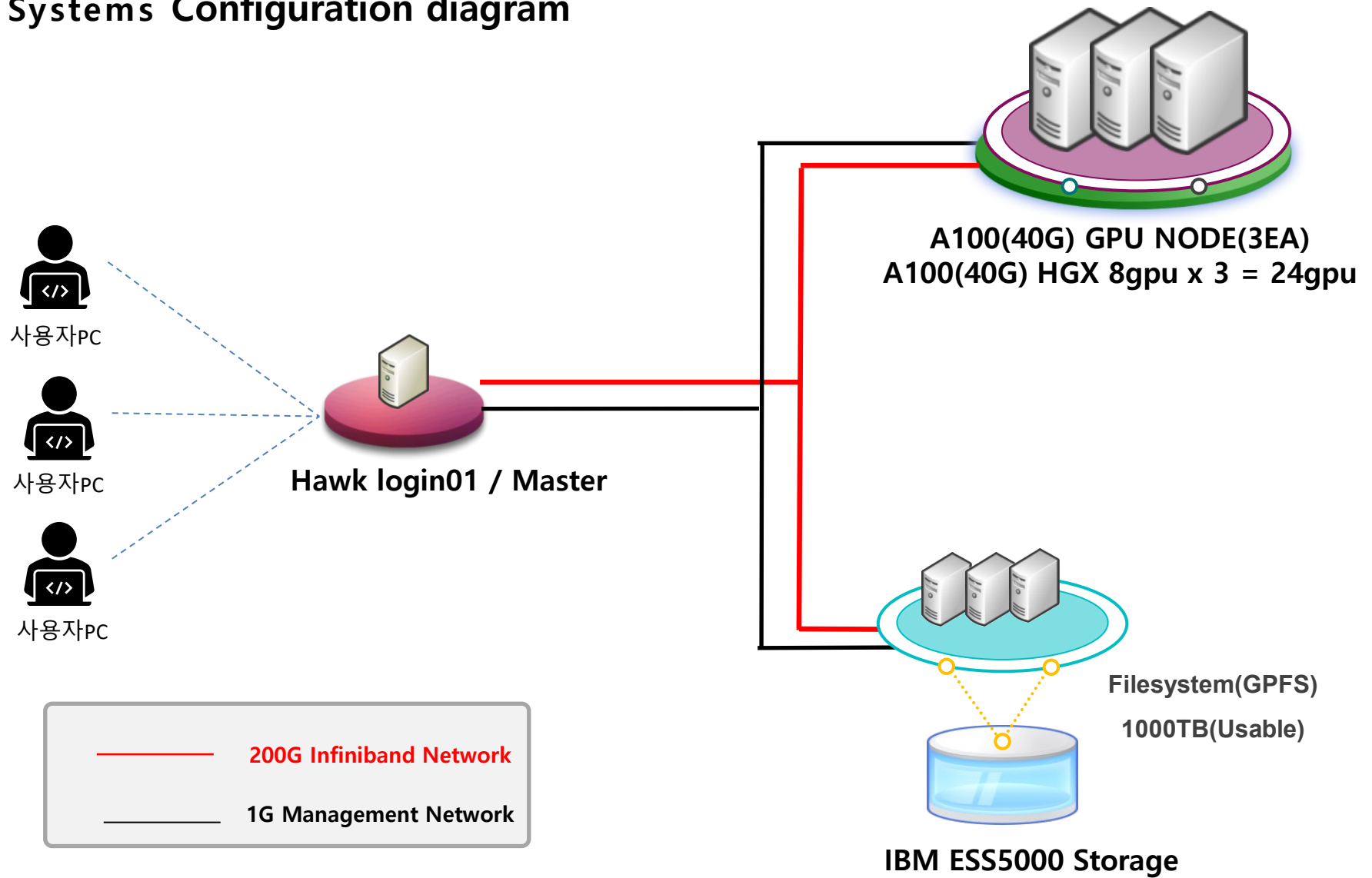
A. Configuration Overview

UNIST Supercomputing Center (USC) supports High Performance Computing (HPC) systems and parallel file systems to support various research. USC has HPC x86_64 Linux clusters such as Tachyon, Leopard, Lion, Falcon, Eagle, and Dumbo, and has Lustre parallel file system and Spectrum Scale (GPFS) as file systems.

The HAWK GPGPU Cluster system consists of 8 A100 (40G) HGX (Nvlink) GPUs per node, with a total of 3 nodes and 24 A100 (40G) GPUs. Each node has 128 cores and 1024 GB of memory. The CPU architecture is AMD EPYC 7742 and the GPU architecture is nVidia A100 (40G). The storage uses a 1PB IBM ESS5000 GPFS parallel file system, and the shared storage and each node are connected by an ultra-high-performance Infiniband 200Gbps network.

The detailed HPC system configuration can be seen in the following figure.

B. Systems Configuration diagram



C. How to access HPC systems

Hawk GPU-HPC system has one login node. Users can access the HPC system through the login node. The connection between the user PC and the login node is made through the SSH protocol (port 2123), and you can use a terminal program (PuTTY, Secure Shell Client, Xmanager, etc.) that enables this.

Hostname	hlogin01
IP	10.0.7.71

※ Systems installed in the UNIST supercomputing center cannot be accessed from outside UNIST. (In special cases, access is allowed only through permitted IPs.)

D. Disk quota

Storage on Hawk GPU-HPC is shared across all nodes via GPFS, where the user's home directory is shared across all nodes. There is no scratch directory on the GPU-HPC system. You can work freely in the assigned home directory space.

Directory name	User Allocated capacity
/home/ <i>UserID</i>	1 TB

O Disk Quota: per account 1TB Hardlimit

E. Hardware Spec (HPC Dumbo Cluster)

i. System Overview

1) GPU Nodes

A) A100 GPU Nodes Spec

구분	Model	CPU	core	Socket	GHz	Memory	DISK	Node #
GPU 노드	NF5488A5	AMD EPYC 7742	64	2	2.25	1024G (32G*32) 3200MHz	1.92TB SSD x2	3
		NVIDIA Tesla A100 GPU 8EA						

구분	Model	용량 (Usable)	DISK	DISK 수량	etc
스토리지	ESS5000	1000TB	NL- SAS 7.2K 16TB	90	hot spare disk 2개를 별도 구성

F. Software

i. Compilers

Compilers	Version
GNU(GCC)	GCC 4.8.5(Linux CentOS 7.8 Default)
Intel compiler	Intel parallel studio 2020(2020.4.9)

ii. Libraries

Classification	Libraries	Libraries built with follow packages.
Parallel libraries	OpenMPI 3.1.6	GCC 4.8.5(Linux CentOS 7.8 Default)
	Mvapich 2.3.4	GCC 4.8.5(Linux CentOS 7.8 Default)
	Intel-impi 2020	Intel Parallel Studio 2020(2020.4.9)
Mathematical libraries	LAPACK 3.9.0	GCC 4.8.5(Linux CentOS 7.8 Default)
	NETCDF 4.6.1	GCC 4.8.5(Linux CentOS 7.8 Default)
	HDF4 4.2.13	GCC 4.8.5(Linux CentOS 7.8 Default)
	HDF5 1.10.2	GCC 4.8.5(Linux CentOS 7.8 Default)
	Intel MKL	Intel Parallel Studio 2020(2020.4.9)
GPU compiler library	Cuda 10.2, nvcc	GCC 4.8.5(Linux CentOS 7.8 Default)
AI Package	Miniconda	
	Jupyterhub	
Container	Nvidia-docker	

2. Slurm Schedule(for Job Control)

A. Overview

In USC, users must use a queue system to execute code. In our USC, we use SLURM (Simple Linux Utility for Resource Management) to manage jobs for effective use and convenience of users.

B. SLURM commands

Commands	Example	Description
sbatch [option...] script	\$sbatch job_script_file_name	submit a job.
squeue	\$squeue	show job status oneself
	\$squeue -u abc	show job status for 'abc' user
	\$squeue -j 12345	show job status for '12345' JobID
sinfo	\$sinfo	show computing node & partition status
	\$sinfo -N -l	show computing node & partition status detail
scancel	\$scancel [job_id]	Cancel a job using its ID
	\$scancel [user_name]	Cancel all jobs from a user

C. Slurm partition information

Partition name	wall clock time	Available cores	Available GPU	User Nodes	Memory CPU cores	Use Account	Remarks
a100_40g	24 hours	Up to group quota	Up to group quota	A100(40G) HGX GPU nodes Hawk[01-03]	Per 1GPU 128GB mem 16 cpu cores	ngpu account	If the wall time is not set or is set to exceed the specified time, the Job will not work. Up to group quota (8GPU)
a100_40g	300 hours	Up to group quota	Up to group quota	A100(40G) HGX GPU nodes Hawk[01-03]	Per 1GPU 128GB mem 16 cpu cores	Lgpu account	If the wall time is not set or is set to exceed the specified time, the Job will not work. Up to group quota (2GPU)

D. sinfo (show computing node & partition status)

\$sinfo

```

PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
gpu_k80    up     infinite   12     idle dumb0[060-071]
cpu_haswell* up     infinite   27     idle dumb0[072-079,082-100]
debug      up     3-00:00:00  1     idle dumb0080

```

\$sinfo -N -l

NODELIST	NODES	PARTITION	STATE	CPUS	S:C:T	MEMORY	TMP_DISK	WEIGHT	AVAIL_FE	REASON
dumb0060	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0061	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0062	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0063	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0064	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0065	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0066	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0067	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0068	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0069	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0070	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0071	1	gpu_k80	idle	16	2:8:1	60000	0	1	Tesla K8	none
dumb0072	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0073	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0074	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0075	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0076	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0077	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0078	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0079	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0080	1	debug	idle	16	2:8:1	30000	0	1	Tesla K8	none
dumb0082	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0083	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0084	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0085	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0086	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0087	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0088	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0089	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0090	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0091	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0092	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0093	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0094	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0095	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0096	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0097	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0098	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0099	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none
dumb0100	1	cpu_haswell*	idle	16	2:8:1	30000	0	1	Xeon Has	none

E. sbatch (submit a job)

```
$sbatch ./job_script.sh
```

[작업 스크립트 예제, GPU MPI 기준]

```
#!/bin/sh
```

```
#SBATCH -J thin_lfts      ## Job Name
```

```
#SBATCH -p a100_40g      ## Partition name
```

```
#SBATCH -N 1              ## node count 총 필요한 컴퓨팅 노드 수
```

```
#SBATCH -n 16             ## 1GPU 당 16core, 만약 1GPU 당 16core 이상 할당 시 무한 Holding
```

```
#SBATCH -o %x.o%j        ## filename of stdout, stdout 파일 명(.o)
```

```
#SBATCH -e %x.e%j        ## filename of stderr, stderr 파일 명(.e)
```

```
#SBATCH --time 2:00:00    ## 최대 작업 시간(Wall Time Clock Limit) 시:분:초
```

```
#SBATCH --gres=gpu:1      ## number of GPU(s) per node
```

```
source /etc/profile.d/modules.sh
```

```
module load intel/20.4.9 mip/impi-20.4.9 ##필요한 Module 로드
```

```
mpirun ./lfts_cuda_single.out inputs    ##해석 실행 코드 입력
```

[Guide to writing a job script]

Information to be passed to slurm is passed by attaching the #SBATCH directive to the job script.

1) Job Name, job name

- If the job name is not specified, slurm assigns one randomly
- If the job name is specified as test, use "#SBATCH --job-name=test" or "#SBATCH -J test".

2) Partition Name, job partition name

- This specifies the partition on which the job will be performed, and the available partition names can be checked with the sinfo command.

If the partition is to be used as gpu_k80, use "#SBATCH --partition=gpu_k80" or "-p gpu_k80".

- ### 3) Amount of required resources
- The total number of processes required to perform the job is defined with the "-n" option, and the number of computing nodes required to perform the job is defined with "-N". ※ -n The number of processes is linked to the number of GPUs and must be written as 16 cores per GPU. For example, when using 4 GPUs, 64 cores or less can be allocated. If this rule is not followed, the job will not be submitted, or even if submitted, calculation will not start (infinite waiting).

4) output

- Define the file name to save the standard output.
- If the file name is test.out, use "#SBATCH --output=test.out" or "#SBATCH -o test.out".

5) error

- Define the file name to save the standard error.
- If the file name is test.err, use "#SBATCH --error=test.err" or "#SBATCH -e test.err".

6) Wall Time Clock Limit

- This refers to the expected task time, and it is safe to set it slightly longer than the actual expected task time. - If the task is not completed within the set time, slurm will forcefully terminate the task when the Wall Time Clock Limit time is exceeded.
- This keyword must be specified in the task submission script file, and can be specified in seconds or hour:minute:second format.
- If the Wall Time Clock Limit is set to 1 hour, use "#SBATCH --time=01:00:00" or "#SBATCH -t 01:00:00".

7) GPU Usage

- It consists of HGX 8GPU per node, and multiples of 2 GPU calculations are efficient, and 2, 4, and 8 GPU calculations are recommended.

F. `squeue` (작업목록 정보 조회)

```
[root@dumbo slurm_test]# squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
92311	debug	test	moasys	R	0:08	2	dumbo088,dumbo089
88915	gpu	GPU_test	moasys	PD	0:00	1	(Resources)
91716	cpu	hello_te	moasys	R	0:00	2	dumbo093,dumbo094
91791	cpu	hello_te	moasys	R	0:00	2	dumbo090,dumbo087
91792	cpu	hello_te	moasys	CG	0:00	2	dumbo090,dumbo091

1) `dumbo` 노드에서 `squeue` 명령어 실행

`squeue - ST`

R - 작업이 컴퓨팅 노드에서 실행 중

PD - 작업이 컴퓨팅 노드에서 대기 중

CG - 작업 완료

`squeue - REASON`

(Resources) - 작업이 컴퓨팅 노드를 사용할 수 있을 때까지 기다리고 대기

(Priority) - 우선 순위가 더 높은 작업이 컴퓨팅 노드를 기다리고 있습니다.

(ReqNodeNotAvail) - 작업에서 요청한 컴퓨팅 노드는 다음과 같은 다양한 이유로 사용할 수 없음

including: cluster downtime, nodes offline, temporary scheduling backlog

G. scontrol (제출된 작업 상세 조회)

scontrol 명령어를 이용하면 제출된 작업의 상세내역을 조회할 수 있습니다.

\$scontrol show job [작업ID]

```
[root@dumbo slurm_test]$ scontrol show job 371
JobId=371 JobName=gpu_mpi
  UserId=moasys(1000) GroupId=moasys(1000) MCS_label=N/A
  Priority=4294901740 Nice=0 Account=g_group QOS=normal
  JobState=FAILED Reason=NonZeroExitCode Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=2:0
  RunTime=00:00:00 TimeLimit=01:00:00 TimeMin=N/A
  SubmitTime=2021-04-02T13:21:30 EligibleTime=2021-04-02T13:21:30
  AccrueTime=2021-04-02T13:21:30
  StartTime=2021-04-02T13:21:30 EndTime=2021-04-02T13:21:30 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2021-04-02T13:21:30
  Partition=gpu_k80 AllocNode:Sid=dumbo:11244
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=dumbo[060-061]
  BatchHost=dumbo060
  NumNodes=2 NumCPUs=8 NumTasks=8 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  TRES=cpu=8,energy=2,node=2,billing=8,gres/gpu=4
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=0 MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=USER Contiguous=0 Licenses=(null) Network=(null)
  Command=/gpfs/home/moasys/slurm_test/gpu_mpi.sh
  WorkDir=/gpfs/home/moasys/slurm_test
  StdErr=/gpfs/home/moasys/slurm_test/gpu_mpi_371.err
  StdIn=/dev/null
  StdOut=/gpfs/home/moasys/slurm_test/gpu_mpi_371.out
  Power=
  TresPerNode=gpu:2
  MailUser=moasys MailType=NONE
```

F. scancel (cancel a job)

\$scancel [작업 ID]

3. Support Apps user-guide

A. Environment Settings

i. Overview

USC has implemented module environment to manage users' environment for all software installed in our HPC systems. Users are required very often to define some variables that affect the environment on each HPCs. And the operation of the program is executed under this environment. These environment variables are used to inform the HPC system the location of execution files, documentation, or related libraries. This would make new user or users unfamiliar with UNIX-like system very annoying. Through the module approach, however, users are no longer painful with configuring user's environment. The scripts(modulefiles) are made by the staff and users simply "load" and "unload" modules to configure the environment, also user can make individual module files.

Environment module approach updates users' environment very easily, especially **PATH**, **LD_LIBRARY_PATH**, **MANPATH** variables, etc.

When your account is made in HPC systems, "dot-files" should be set up for user's environment(default – bash shell). You can modify this "dot-files" to configure your environment firsthand. But, there is a way to set your environment up more conveniently.

ii. Module commands

To know how to use environment module approach, type the following:

```
$ module help
```

"help" option displays as below. (not full result, select useful options)

```
Available SubCommands and Args:
+ add|load      modulefile [modulefile ...]
+ rm|unload     modulefile [modulefile ...]
+ switch|swap   [modulefile1] modulefile2
```

```
+ display|show      modulefile [modulefile ...]
+ avail             [modulefile [modulefile ...]]
+ purge
+ list
```

\$ module list

prints a list of the currently loaded modulefile.

\$ module avail

lists all the modulefiles which are available to be loaded.

\$ module purge

unloads all loaded modulefiles currently.

\$ module load *modulefile*

loads *the modulefile*.

\$ module unload *modulefile*

unload *the modulefile*.

\$ module switch modulefile_old modulefile_new

switches *the modulefile_old* with *modulefile_new*.

\$ module show *modulefile*

displays how the *modulefile* changes the environment such as PATH, LD_LIBRARY_PATH, MANPATH, etc.

B. Conda Environment settings(using miniconda)

i. Overview

To use the GPU-based AI Framework, you need to use the CONDA environment. Users can use the AI Framework in the module installed by default, and can configure their own environment in a personal folder.

ii. Module commands

Below is how to load the conda environment in module.

```
$ module list
```

```
----- /apps/Modules/modulefiles/conda_packages -----  
Conda/pytorch      conda/tensorflow      conda/tensorflow-gpu
```

```
$ module load conda/tensorflow-gpu
```

The installed module is the most stable version currently installed by conda-forge. However, in this case, the latest version of the framework

You can't use it, so you can personally install the files you need.

In the case of modules, the environment imported from python can be used as it is, so the environment does not change like conda load.

Since python in the environment loaded from the module is installed according to the required version, execute the following command to check the information of conda installed in each version.

```
$ CONDA_HOME=/apps/application/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh  
$ conda env list
```

iii. Using conda environments in your user folder

To load the required conda environment from your user folder, run the following command:.

```
$ CONDA_HOME=/apps/applications/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh
```

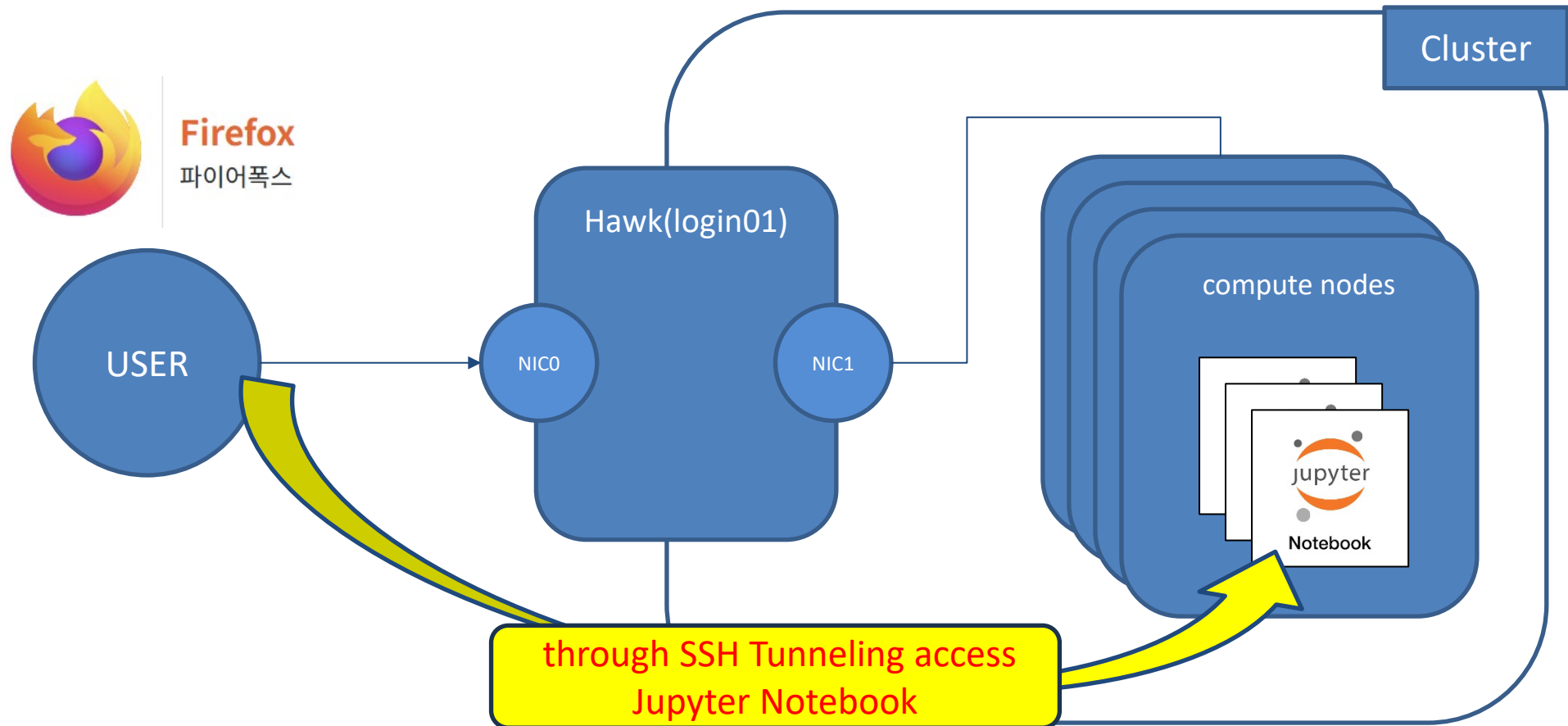
After performing the Conda environment setup, configure the conda environment in each user's home folder.

```
$ CONDA_HOME=/apps/applications/miniconda3  
$ source $CONDA_HOME/etc/profile.d/conda.sh  
$ conda create -n tf_2.2_gpu  
Collecting package metadata (current_repodata.json): done  
Solving environment: done  
  
## Package Plan ##  
  
environment location: /home/moasys/.conda/envs/tf_2.2_gpu  
  
Proceed ([y]/n)? y  
  
Preparing transaction: done  
Verifying transaction: done  
Executing transaction: done  
#  
# To activate this environment, use  
#  
# $ conda activate tf_2.2_gpu  
#  
# To deactivate an active environment, use  
#  
# $ conda deactivate
```

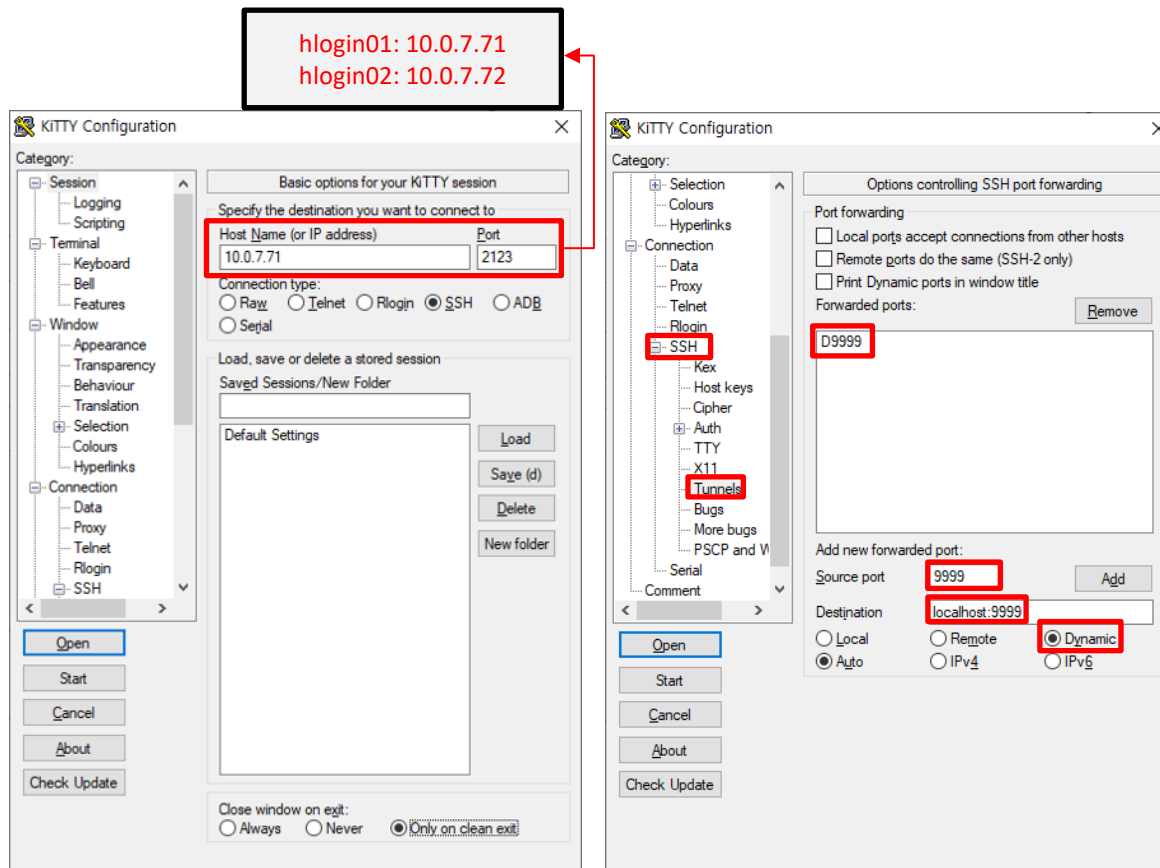
C. Jupyter Notebook Settings

i. SSH Tunneling setup overview

- Instructions on how to set up a Jupyter Notebook on a User PC web browser via SSH tunneling so that it can be accessed and used by connecting to the UNIST Supercomputing Center system.



ii. SSH tunneling Settings (putty standard settings)



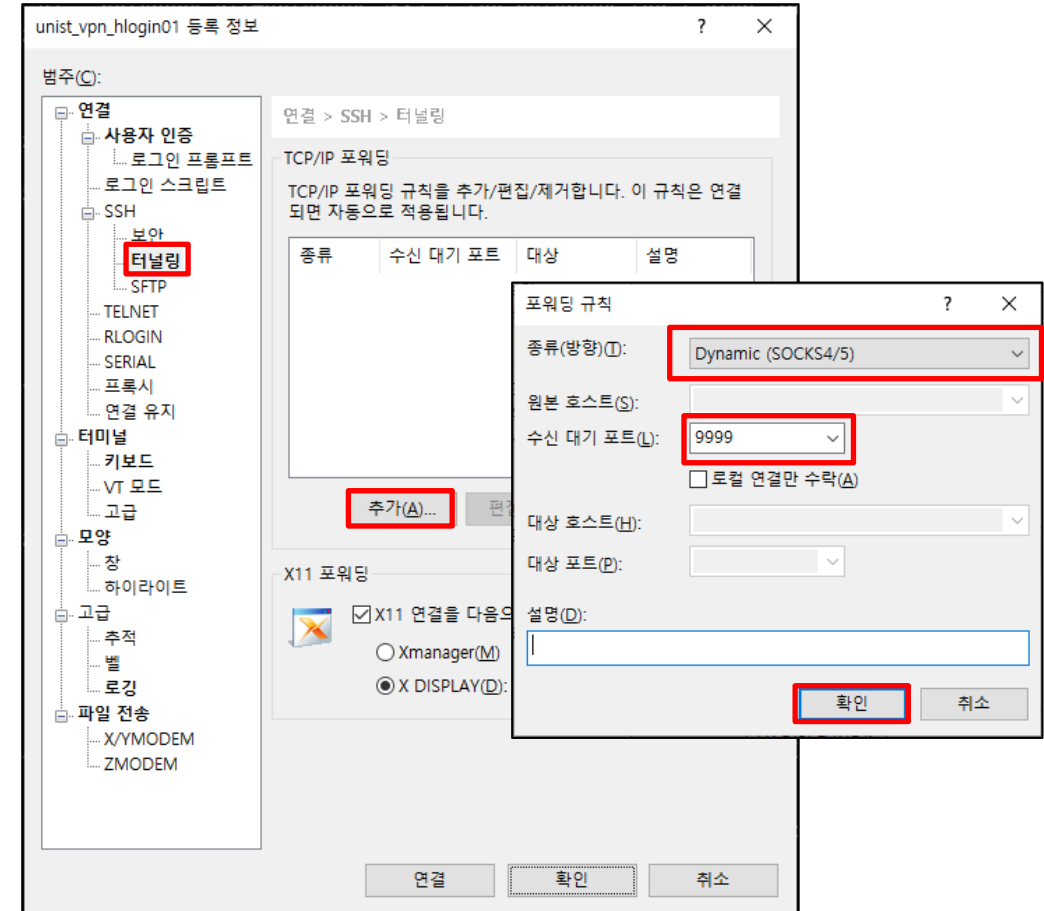
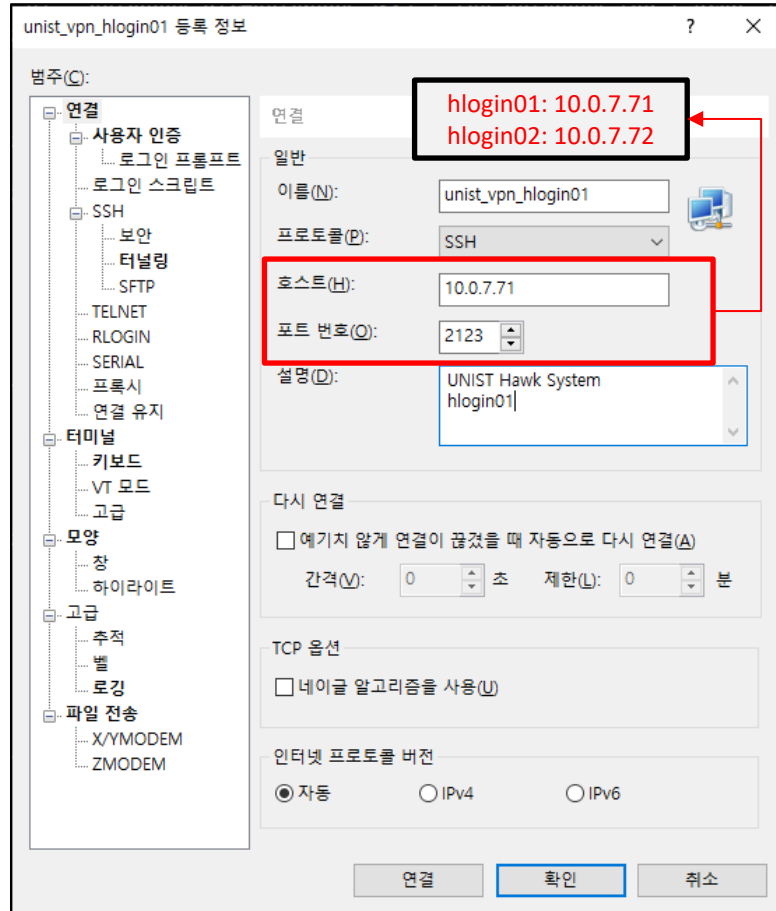
hlogin01: 10.0.7.71
hlogin02: 10.0.7.72

- Left menu SSH → Tunnels
- Source port: 9999
- Destination: localhost:9999
- Dynamic select
- Add Add items with buttons

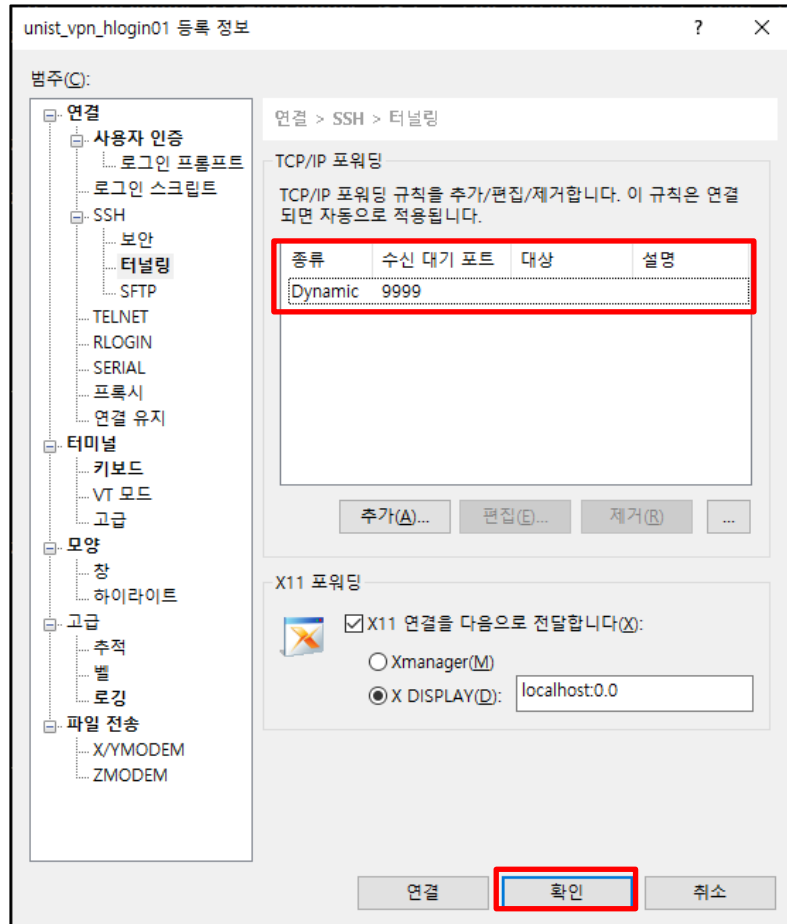
Note that the example images are largely the same configuration as kitty or putty.

- SSH tunneling setup complete
- After connecting to the server with the completed profile and logging in, the SSH tunneling function is activated.

iii. SSH tunneling Settings (for Xshell) 1/2



iii. SSH tunneling Settings (for Xshell) 2/2

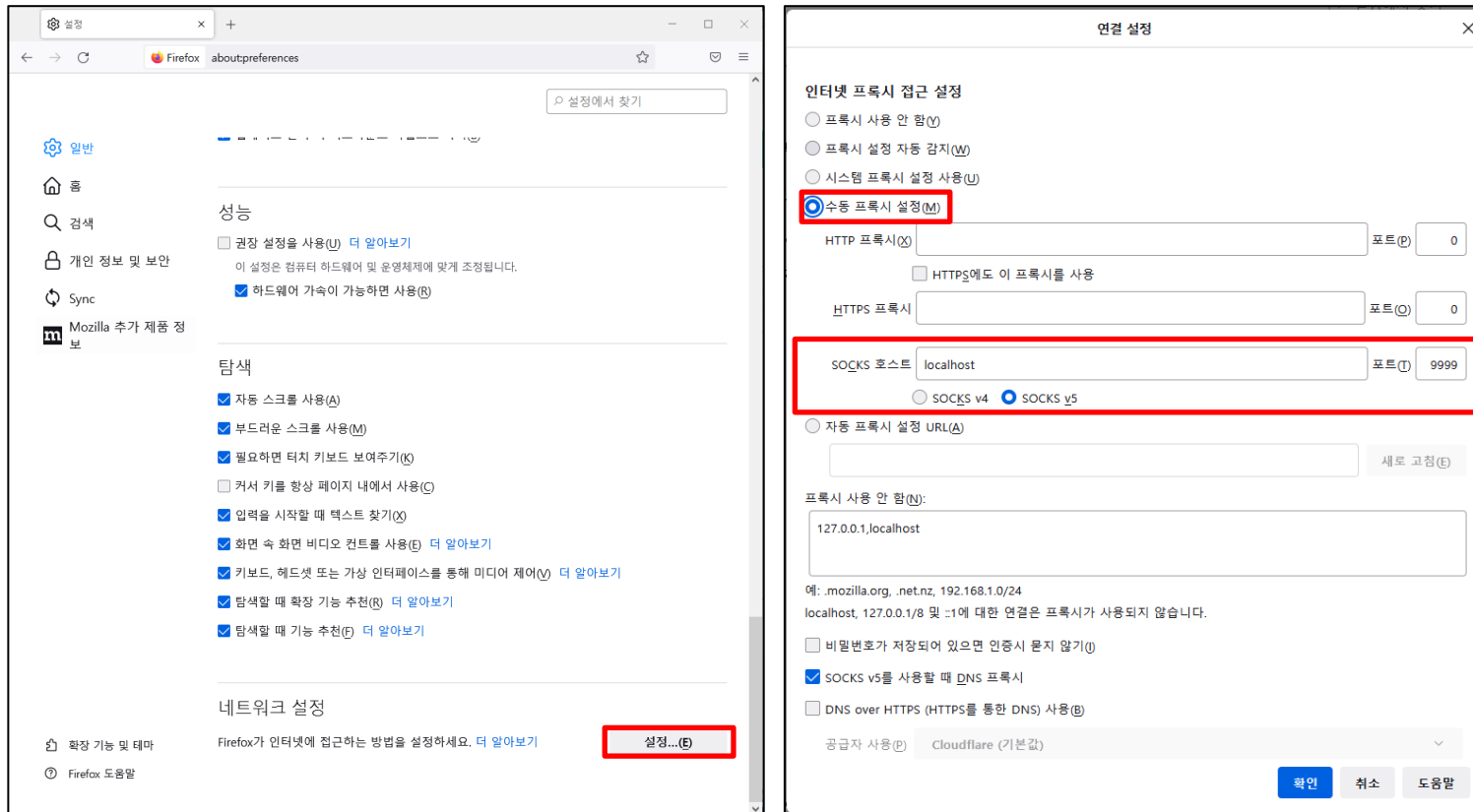


- SSH tunneling setup complete
- After connecting to the server with the completed profile and logging in, the SSH tunneling function is activated.

iv. Browser proxy settings on my PC (Windows/Mac) (Firefox standard)



Firefox
파이어폭스

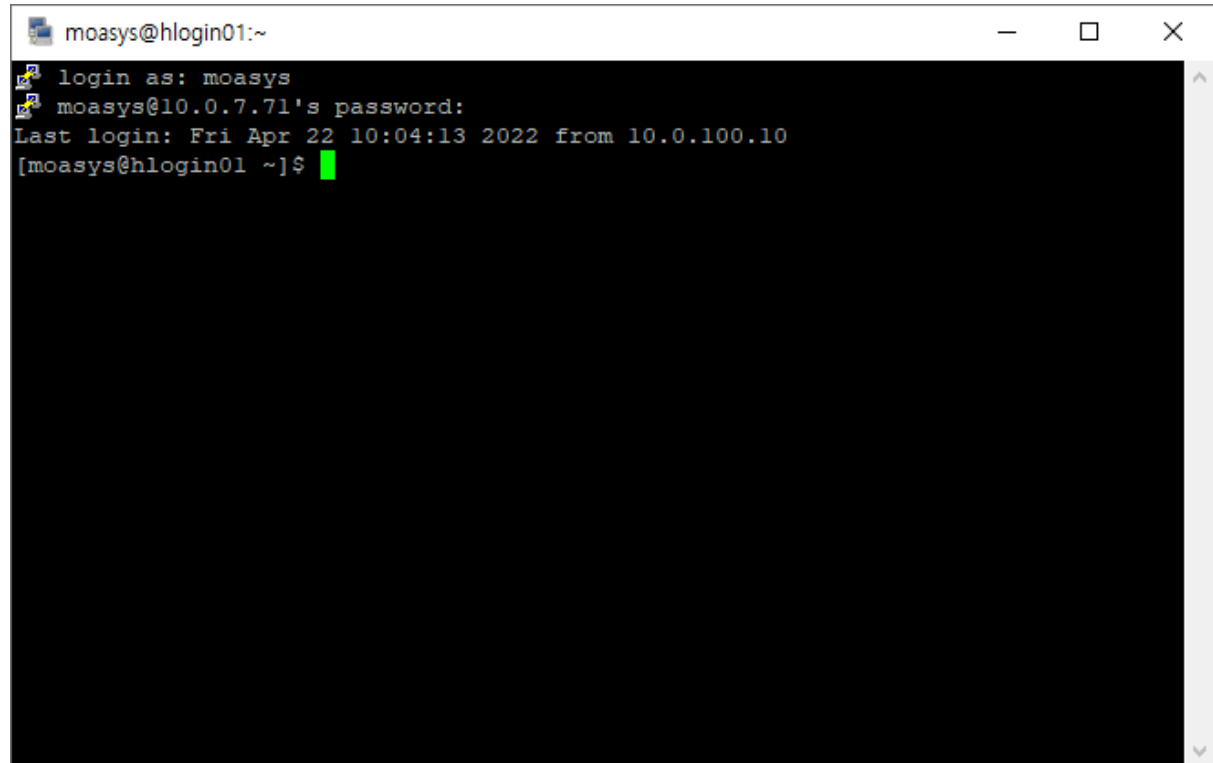
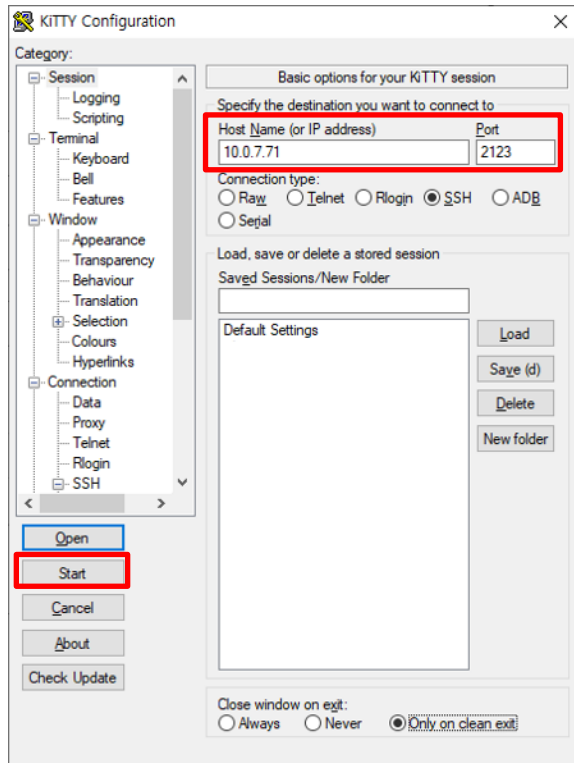


- 수동 프록시 설정 → SOCKS 호스트 설정
- SSH 터널링 설정에서 지정한 포트 입력
→ localhost:9999

v. After setting up tunneling, connect to UNIST supercomputer (Linux) and set up 1/3

How to run and connect to Jupyter Notebook

- SSH Tunneling Connect to the login server (hlogin01 (10.0.7.71)) with the configured client
SSH tunneling is enabled upon successful login



XSHELL
PLUS

v. After setting up tunneling, connect to UNIST supercomputer (Linux) and set up 2/3

How to run and connect to Jupyter Notebook

- Jupyter Notebook script Example - **myjupyter.sh** Instructions for use on the next page

```
#!/bin/sh
#SBATCH - J myjupyter
#SBATCH - p gpu
#SBATCH - N 1
#SBATCH - n 16
#SBATCH - o %x.%j
#SBATCH - e %x.%j
#SBATCH -- time 12:00:00
#SBATCH -- gres=gpu:1
#SBATCH -- comment etc
#SBATCH -- nodelist=hawk03
```

Output 파일명: <JobName>.<JobID>
본 스크립트에서는 myjupyter.<JobID>

활성화할 jupyter notebook conda
환경 이름 지정

```
source /apps/applications/miniconda3/bin/activate notebook
```

```
jupyter notebook --no-browser --ip=$(hostname -i) --port=5$(id -u) --notebook-dir=$HOME/notebook
```

jupyter notebook 작
업 경로 지정

v. After setting up tunneling, connect to UNIST supercomputer (Linux) and set up 3/3

How to run and connect to Jupyter Notebook

- Submit the batch job script written in the previous step in the running terminal

```
$ sbatch myjupyter.sh
```

```
Submitted batch job 1234
```

앞 페이지 스크립트 예제 참조

- After confirming that the batch job has been executed normally, copy the connection URL.

```
$ squeue -u $USER
```

```
$ cat myjupyter.1234
```

```
...
```

```
...
```

To access the notebook, open this file in a browser:

```
file:///gpfs/home/moasys/.local/share/jupyter/runtime/nbserver-32164-open.html
```

Or copy and paste one of these URLs:

```
http://172.30.0.13:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6
```

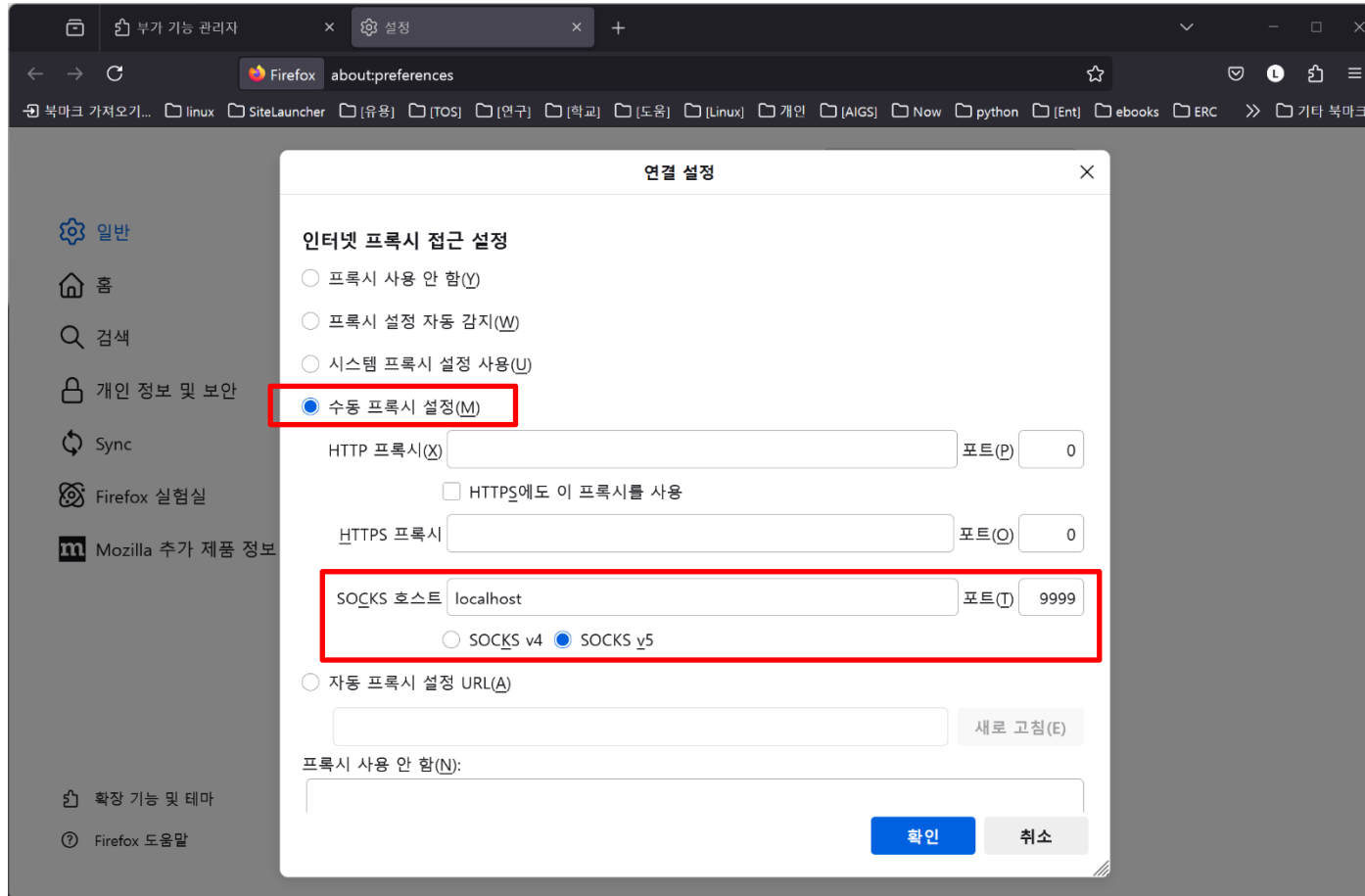
```
or http://127.0.0.1:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6
```

Or copy and paste...바로 아래줄의
URL 및 토큰 복사.
127.0.0.1은 loopback IP이므로 원격
웹 브라우저 접속 시 사용 불가

vi. After setting up tunneling, access the browser on my PC (explanation based on Firefox)1/2

How to run and connect to Jupyter Notebook

- Setting up SOCKS v5 proxy in Firefox
- 'Settings' -> 'General' -> 'Network Settings... (E)'

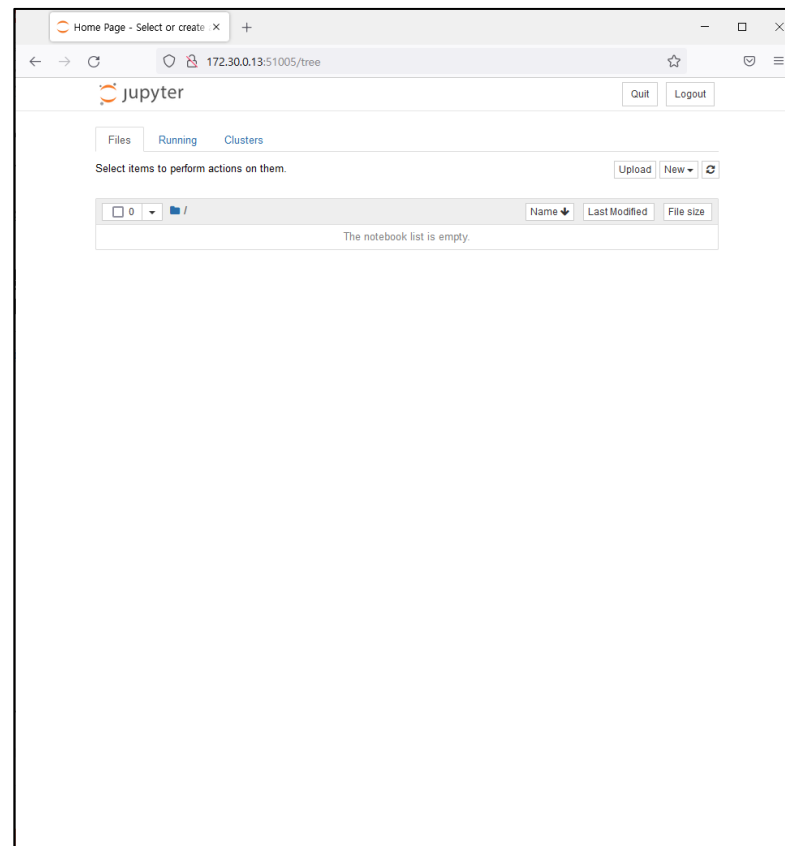
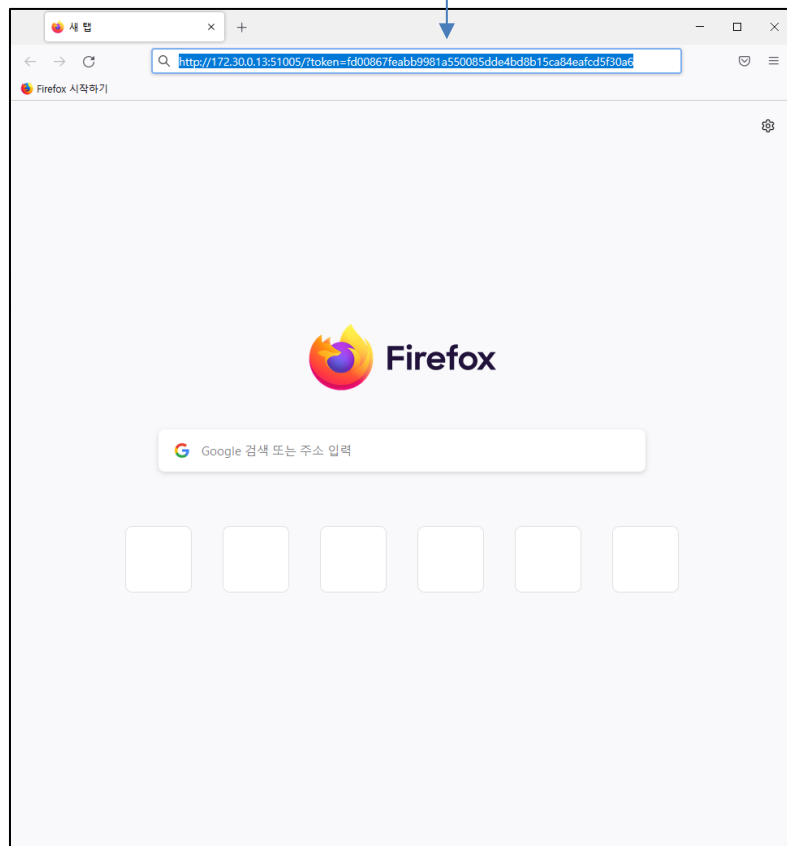


vi. After setting up tunneling, access the browser on my PC (explanation based on Firefox)2/2

How to run and connect to Jupyter Notebook

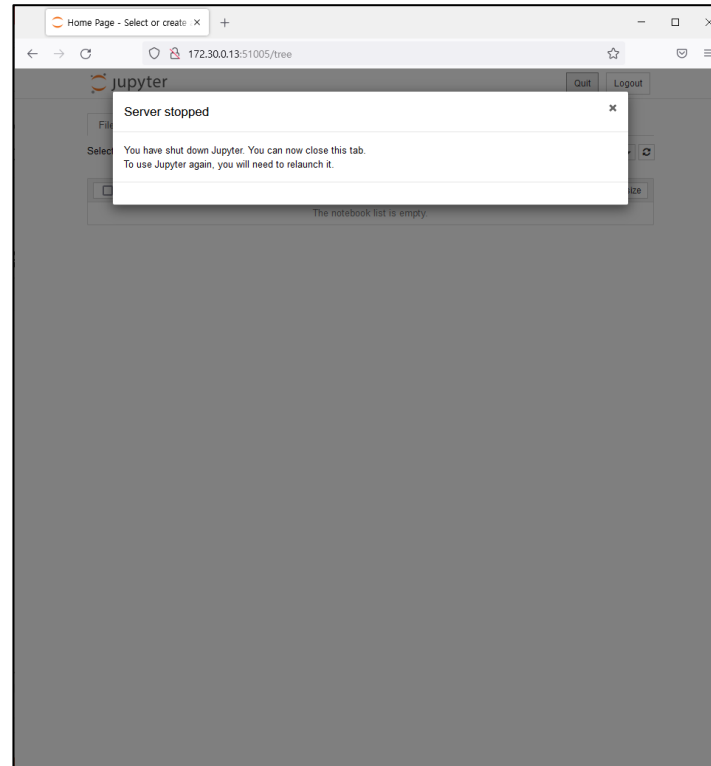
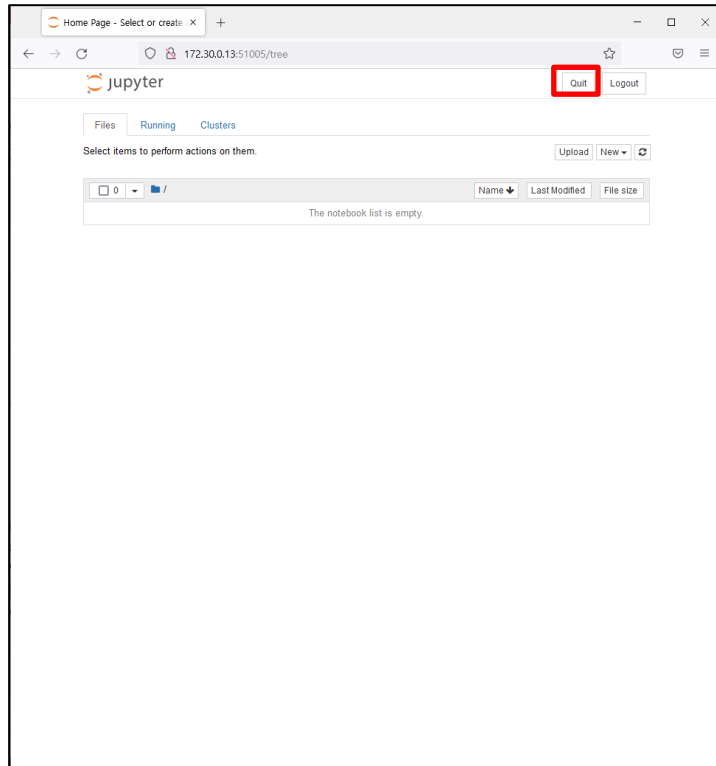
- After launching the browser with the SOCKS v5 proxy set, connect by pasting the Jupyter Notebook URL and token copied in the previous step into the address bar.

<http://172.30.0.13:51005/?token=fd00867feabb9981a550085dde4bd8b15ca84eafcd5f30a6>



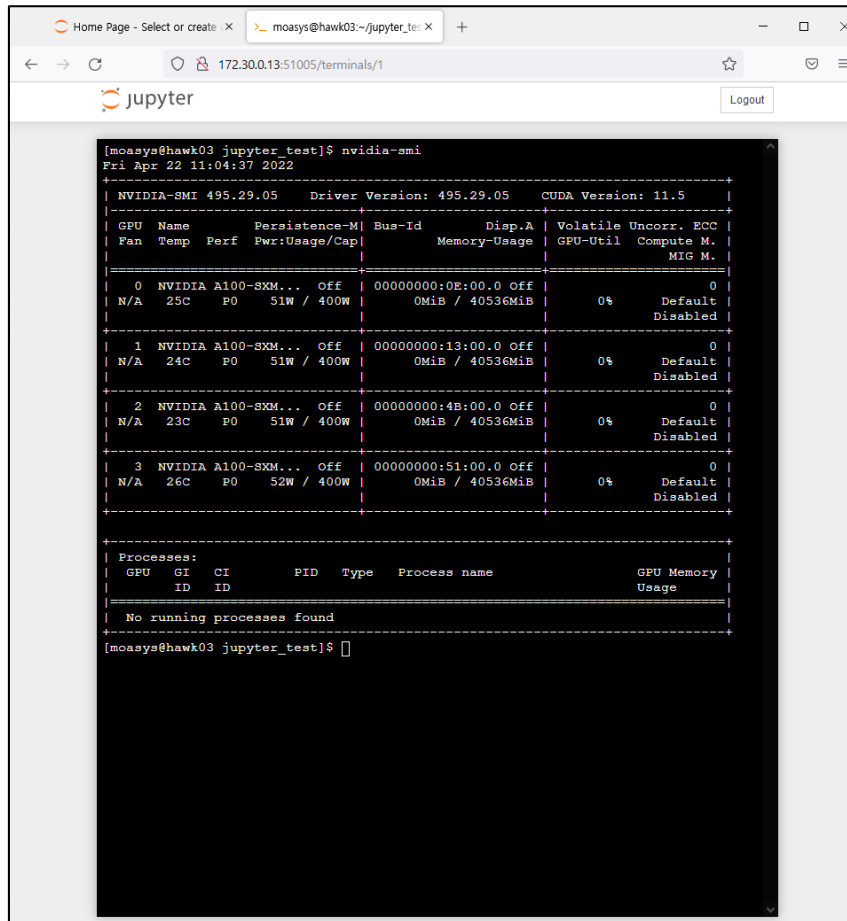
vii. Checking and exiting the Jupyter Notebook environment 1/2

- When you are done using it, you must explicitly exit it with the Quit button. If you close the web browser without explicitly exiting, the Jupyter Notebook will not be terminated and will remain running on the compute node, continuing to occupy system resources.



vii. Checking and exiting the Jupyter Notebook environment 2/2

- There is also a way to terminate the batch job itself that is running Jupyter Notebook in the terminal, but please note that there is a risk of data loss.
- (Other) Only available within the resource limit allocated (scheduler) through the `myjupyter.sh` script (GPU resources)



```
[moasys@hawk03 jupyter_test]$ nvidia-smi
Fri Apr 22 11:04:37 2022

+-----+
| NVIDIA-SMI 495.29.05      Driver Version: 495.29.05   CUDA Version: 11.5     |
+-----+
| GPU   Name                               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====================================+-----+
| 0  NVIDIA A100-80GB...    Off          | 00000000:0E:00:0 Off |           0          |
| N/A   25C   P0   51W / 400W      0MiB / 40536MiB | 0%      Default |
|                               |                      | Disabled  MIG M.    |
+-----+
| 1  NVIDIA A100-80GB...    Off          | 00000000:13:00:0 Off |           0          |
| N/A   24C   P0   51W / 400W      0MiB / 40536MiB | 0%      Default |
|                               |                      | Disabled  MIG M.    |
+-----+
| 2  NVIDIA A100-80GB...    Off          | 00000000:4B:00:0 Off |           0          |
| N/A   23C   P0   51W / 400W      0MiB / 40536MiB | 0%      Default |
|                               |                      | Disabled  MIG M.    |
+-----+
| 3  NVIDIA A100-80GB...    Off          | 00000000:51:00:0 Off |           0          |
| N/A   26C   P0   52W / 400W      0MiB / 40536MiB | 0%      Default |
|                               |                      | Disabled  MIG M.    |
+-----+

+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                      GPU Memory |
| ID   ID   ID              |              | Usage                     |
+-----+
| No running processes found |
+-----+

[moasys@hawk03 jupyter_test]$
```


D. Nvidia-Docker

i. Nvidia-Docker Script example

```
#!/bin/bash
#SBATCH -J docker_job          # job name
#SBATCH -p gpu                 # Specify the Partition (=queue) to use
#SBATCH -N 1                   # Specify the number of calculation nodes to use
#SBATCH -n 16                  # Total number of processes to use
                               (CPU:GPU 16:4 It is good to allocate it as a ratio, but it is optional.)
#SBATCH -o $x.o%j              # Standard output file format
#SBATCH -e %x.e%j              # standard error file format
#SBATCH --gres=gpu:4           # Number of GPUs to use per node
#SBATCH --time 2:00:00         # walltim
```

module purge

```
CONT='Ubuntu' # Change the docker image name to use
MOUNT=/mnt # Specify mount point inside docker image
```

```
# Since only one docker job can be performed per SLURM job ID,
# nvidia-docker can only be run once in one SLURM job script.
```

**nvidia-docker **

```
    run -v $PWD:$MOUNT -w $MOUNT \ # Mount the host's job submission path inside the docker image and specify it as the working directory
    $CONT \ # docker image name to use
    mpirun -np 4 mpi.exe # The executable statement is modified to fit the user's docker image. If it is an MPI task, specify the
number of MPI Processes to be executed in the execution statement.
```

ii. Nvidia-Docker Interactive mode(1/2)

Interactive working example (docker)

1) To perform interactive work using the docker container environment, resources must first be allocated through the SLURM scheduler.

```
$ salloc --partition=gpu -N=< number of nodes > -n < Total number of CPUs to use > --gres=gpu:< Number of GPUs per node > --time=02:00:00
```

The maximum available resources on one node are 16 CPUs and 4 GPUs, and memory is allocated in proportion to the number of CPU cores. (2048 MB per core)

It is recommended to allocate CPU:GPU at a ratio of 16:4, but it is not mandatory and you can allocate as much as necessary for the task.

When working inside a Docker image, work can only be performed within the resource limit allocated by SLURM, and work can only be performed within the allocated memory limit.

If you perform any operation, the system may force the shutdown.

Additionally, it is recommended that docker operations be performed on a single node.

2) Enter the calculation node allocated through the SLURM scheduler.

```
$ srun --pty bash
```

3) After moving to the working directory where the executable binary or input file is located, you can enter the docker image in interactive mode and perform work.

Since the resource settings allocated from SLURM are automatically applied to docker tasks, separate GPU-related settings (--ngpus) are not required. Please note that the use of parameters such as --ngpus and --name is restricted.

```
$ cd < working directory >
$ nvidia-docker run -it -v $PWD:/mnt -w /mnt <docker image name >
I have no name!@< Container ID >:/mnt$ mpirun ./mpi.exe
.....
.....
I have no name!@< Container ID >:/mnt$
```

ii. Nvidia-Docker Interactive mode(2/2)

4) When performing a docker operation in interactive mode, the first "nvidia-docker run -it ..." After entering the image once with the command "nvidia-docker run -it ..." If you try to perform the command, the following error may occur:

```
A container executed in the same SLURM job already exists  
please remove user001_1234 (in hawk01)
```

When resources for a computation node are allocated through SLURM, only one docker task is allowed to be performed in one SLURM task for resource management.

To perform another docker task in one SLURM task (interactive mode), you must first delete the docker task that was terminated immediately before.

```
$ nvidia-docker rm <USER ID>_<SLURM_JOB_ID>
```

5) If you need to access multiple directories on a host, you can mount multiple host directories by using the -v argument multiple times as follows:

```
$ cd < working directory >  
$ nvidia-docker run -it \  
-v $PWD:/mnt  
-v <호스트 디렉터리1>:/< Path to mount inside image 1 >  
-v <호스트 디렉터리2>:/< Path to mount inside image 2 >  
-w /mnt <docker image name >  
  
I have no name!@< Container ID >:/mnt$
```

4. 작업 스크립트 예제

i. 작업 스크립트 예제(요약)

1. CPUs serial 작업

```
#!/bin/sh
#SBATCH -J Serial_job
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell

./serial.exe
```

2. CPUs openmp 작업

```
#!/bin/sh
#SBATCH -J OpenMP_job
#SBATCH --time=01:00:00
#SBATCH -N 1 # number of node
#SBATCH -n 1
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell

export OMP_NUM_THREADS=10

./omp.exe
```

3. CPUs mpi 작업

```
#!/bin/sh
#SBATCH -J MPI_job
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 2 # number of node
#SBATCH -n 8 # total process

srun ./mpi.exe
```

4. CPUs hybrid 작업

```
#!/bin/sh
#SBATCH -J omp_mpi
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 1 # number of node
#SBATCH -n 2

export OMP_NUM_THREADS=10

srun ./a.out
```



작업 스크립트 예제(요약)

5. CPUs Array 작업

```
#!/bin/sh
#SBATCH -J omp_mpi
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p cpu_haswell
#SBATCH -N 1                # number of node
#SBATCH -n 2
#SBATCH --array 1-10

export OMP_NUM_THREADS=10

srun ./a.out
```

6. GPU serial job

```
#!/bin/sh
#SBATCH -J gpu_k80
#SBATCH --time=01:00:00
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -p gpu
#SBATCH --gres=gpu:2 # using 2 gpus

./serial.exe
```

7. GPU openmp job

```
#!/bin/sh
#SBATCH -J gpu_omp
#SBATCH --time=01:00:00
#SBATCH -p gpu_k80
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -N 1 # number of node
#SBATCH -n 1
#SBATCH --gres=gpu:2 # using 2 gpus

export OMP_NUM_THREADS=10

./omp.exe
```

8. GPU mpi job

```
#!/bin/sh
#SBATCH -J gpu_k80
#SBATCH --time=01:00:00
#SBATCH -p gpu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
#SBATCH -N 2 # number of node
#SBATCH -n 8 # total process
#SBATCH --gres=gpu:2 # using 2 gpus

srun ./mpi.exe
```

ii. 작업 스크립트 예제(상세)

A. 작업 스크립트 예제 (CPU)

1) CPU Serial Job

```
#!/bin/bash
#SBATCH -J cpu_Serial          # 작업명
#SBATCH -p gpu                # 사용할 Partition(=queue) 지정
#SBATCH -N 1                  # 사용할 계산노드 수 지정
#SBATCH -n 1                  # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j             # 표준출력 파일 형식
#SBATCH -e %x.e%j             # 표준에러 파일 형식
#SBATCH --gres=gpu:0           # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00         # walltime 설정

module purge
module load intel/20.4.9

./serial.exe
```

2) CPU OpenMP Job

```
#!/bin/bash
#SBATCH -J cpu_OpenMP          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 1                   # 사용할 전체 프로세스 수
#SBATCH -c 8                   # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j              # 표준출력 파일 형식
#SBATCH -e %x.e%j              # 표준에러 파일 형식
#SBATCH --gres=gpu:0            # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00          # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load intel/20.4.9

./omp.exe
```

3) CPU MPI Job

```
#!/bin/bash
#SBATCH -J cpu_MPI          # 작업명
#SBATCH -p gpu              # 사용할 Partition(=queue) 지정
#SBATCH -N 1                # 사용할 계산노드 수 지정
#SBATCH -n 8                # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j           # 표준출력 파일 형식
#SBATCH -e %x.e%j           # 표준에러 파일 형식
#SBATCH --gres=gpu:0         # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

module purge
module load intel/20.4.9 mpi/impi-20.4.9

srun ./mpi.exe
```


4) CPU Hybrid Job

```
#!/bin/bash
#SBATCH -J cpu_Hybrid          # 작업명
#SBATCH -p gpu                # 사용할 Partition(=queue) 지정
#SBATCH -N 1                  # 사용할 계산노드 수 지정
#SBATCH -n 2                  # 사용할 전체 프로세스 수
#SBATCH -c 8                  # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j             # 표준출력 파일 형식
#SBATCH -e %x.e%j             # 표준에러 파일 형식
#SBATCH --gres=gpu:0           # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00         # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load intel/20.4.9 mpi/impi-20.4.9

srun ./hybrid.exe
```

B. 작업 스크립트 예제 (GPU)

1) GPU Serial Job

```
#!/bin/bash
#SBATCH -J gpu_Serial      # 작업명
#SBATCH -p gpu             # 사용할 Partition(=queue) 지정
#SBATCH -N 1               # 사용할 계산노드 수 지정
#SBATCH -n 1               # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j          # 표준출력 파일 형식
#SBATCH -e %x.e%j          # 표준에러 파일 형식
#SBATCH --gres=gpu:1        # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

module purge
module load gcc/4.8.5 cuda/11.4

./serial.exe
```

2) GPU OpenMP Job

```
#!/bin/bash
#SBATCH -J gpu_OpenMP          # 작업명
#SBATCH -p gpu                 # 사용할 Partition(=queue) 지정
#SBATCH -N 1                   # 사용할 계산노드 수 지정
#SBATCH -n 1                   # 사용할 전체 프로세스 수
#SBATCH -c 8                   # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j              # 표준출력 파일 형식
#SBATCH -e %x.e%j              # 표준에러 파일 형식
#SBATCH --gres=gpu:2            # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00          # walltime 설정

export OMP_NUM_THREADS=8

module purge
module load gcc/4.8.5 cuda/11.4

./omp.exe
```

3) GPU MPI Job

```
#!/bin/bash
#SBATCH -J cpu_MPI          # 작업명
#SBATCH -p gpu              # 사용할 Partition(=queue) 지정
#SBATCH -N 1                # 사용할 계산노드 수 지정
#SBATCH -n 8                # 사용할 전체 프로세스 수
#SBATCH -o $x.o%j           # 표준출력 파일 형식
#SBATCH -e %x.e%j           # 표준에러 파일 형식
#SBATCH --gres=gpu:8         # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

module purge
module load gcc/4.8.5 cuda/11.4 cudampi/openmpi-4.1.1

srun ./mpi.exe
```

4) GPU Hybrid Job

```
#!/bin/bash
#SBATCH -J cpu_MPI          # 작업명
#SBATCH -p gpu              # 사용할 Partition(=queue) 지정
#SBATCH -N 1                # 사용할 계산노드 수 지정
#SBATCH -n 8                # 사용할 전체 프로세스 수
#SBATCH -c 8                # 사용할 OpenMP 스레드 수
#SBATCH -o $x.o%j          # 표준출력 파일 형식
#SBATCH -e %x.e%j          # 표준에러 파일 형식
#SBATCH --gres=gpu:8        # 노드당 사용할 GPU 수
#SBATCH --time 2:00:00      # walltime 설정

export OMP_NUM_THREADS=8
module purge
module load gcc/4.8.5 cuda/11.4 cudampi/openmpi-4.1.1

srun ./hybrid.exe
```